STAT 238 – Bayesian Statistics

Homework 4

Thomas Lee

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, chi2
from scipy.linalg import solve, svd as lsvd, solve_triangular
from scipy.special import gammaln
from sklearn.linear_model import LogisticRegression, Ridge, RidgeCV
from sklearn.model_selection import KFold
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')
```

Problem 1

(a) — Newton's Method for Logistic Regression MLE

The logistic regression model assumes

$$P(y_i = 1 \mid x_i) = \sigma(x_i'\beta) := \frac{e^{x_i'\beta}}{1 + e^{x_i'\beta}},$$

giving log-likelihood

$$\ell(\beta) = \sum_{i=1}^{n} [y_i x_i'\beta - \log(1 + e^{x_i'\beta})].$$

The gradient and Hessian are

$$\nabla \ell = X'(y - p), \qquad H = -X'WX,$$

where $p_i = \sigma(x_i'\beta)$ and $W = \mathrm{diag}(p_i(1 - p_i))$.

Newton's update is $\beta \leftarrow \beta + (X'WX)^{-1}X'(y - p)$.

```
spam = pd.read_csv("spam7.csv")

y = (spam["yesno"] == "y").astype(float).values
X_raw = spam[["crl.tot", "dollar", "bang", "money", "n000", "make"]].values
X_cov = np.column_stack([
    np.log(X_raw[:, 0]),
    np.log(X_raw[:, 1] + 0.001),
    np.log(X_raw[:, 2] + 0.001),
    np.log(X_raw[:, 3] + 0.001),
    np.log(X_raw[:, 4] + 0.001),
    np.log(X_raw[:, 5] + 0.001),
])
X = np.column_stack([np.ones(len(y)), X_cov])
n, p = X.shape

def sigmoid(z):
    return np.where(z >= 0, 1 / (1 + np.exp(-z)), np.exp(z) / (1 + np.exp(z)))

def newton_logistic(X, y, tol=1e-8, max_iter=200):
    beta = np.zeros(X.shape[1])
    for _ in range(max_iter):
        p_vec = sigmoid(X @ beta)
        W = p_vec * (1 - p_vec)
        grad = X.T @ (y - p_vec)
        H = -(X.T * W) @ X
        step = np.linalg.solve(-H, grad)
        beta += step
        if np.max(np.abs(step)) < tol:
```

```
            break
    return beta, H

beta_mle, H_mle = newton_logistic(X, y)
print("Newton MLE coefficients:")
labels = ["Intercept", "log(crl.tot)", "log(dollar)", "log(bang)",
          "log(money)", "log(n000)", "log(make)"]
for lbl, b in zip(labels, beta_mle):
    print(f"  {lbl:<18}: {b:+.4f}")
```

```
Newton MLE coefficients:
  Intercept         : +4.1195
  log(crl.tot)      : +0.3023
  log(dollar)       : +0.3259
  log(bang)         : +0.4098
  log(money)        : +0.3456
  log(n000)         : +0.1895
  log(make)         : -0.1142
```

```
# Verify with sklearn (C large = near-flat prior)
lr = LogisticRegression(penalty=None, solver="lbfgs", max_iter=1000)
lr.fit(X_cov, y)
beta_sklearn = np.concatenate([[lr.intercept_[0]], lr.coef_[0]])
print("\nSklearn coefficients:")
for lbl, b in zip(labels, beta_sklearn):
    print(f"  {lbl:<18}: {b:+.4f}")

max_diff = np.max(np.abs(beta_mle - beta_sklearn))
print(f"\nMax absolute difference: {max_diff:.2e}")
```

```
Sklearn coefficients:
  Intercept         : +4.1189
  log(crl.tot)      : +0.3022
  log(dollar)       : +0.3259
  log(bang)         : +0.4098
  log(money)        : +0.3455
  log(n000)         : +0.1895
  log(make)         : -0.1142

Max absolute difference: 5.80e-04
```

The Newton MLE and sklearn agree to within numerical tolerance, confirming the implementation is correct. All six log-transformed features have positive coefficients, consistent with the intuition that higher capital letter usage, dollar signs, and spam-indicator words all increase the probability of spam.

(b) — Laplace Posterior Normal Approximation

With the flat prior $\beta_j \overset{\text{i.i.d.}}{\sim} \text{Uniform}(-\infty, \infty)$, the log-posterior equals the log-likelihood, so the posterior Hessian equals $H(\hat{\beta})$ and the Laplace approximation is

$$\beta \mid \text{data} \approx N\Big(\hat{\beta},\ (X'\hat{W}X)^{-1}\Big)\,.$$

The posterior standard errors are $\sqrt{\text{diag}\Big[(X'\hat{W}X)^{-1}\Big]}$.

```
Fisher_info = -(H_mle)   # X'WX at beta_mle
post_cov = np.linalg.inv(Fisher_info)
post_se = np.sqrt(np.diag(post_cov))

# sklearn SEs via Hessian at sklearn solution
p_sk = sigmoid(X @ beta_sklearn)
W_sk = p_sk * (1 - p_sk)
Fisher_sk = (X.T * W_sk) @ X
se_sklearn = np.sqrt(np.diag(np.linalg.inv(Fisher_sk)))

print(f"{'':18} {'Laplace SE':>12} {'Sklearn SE':>12}")
for lbl, se1, se2 in zip(labels, post_se, se_sklearn):
    print(f"  {lbl:<18} {se1:12.4f} {se2:12.4f}")
```

```
                    Laplace SE   Sklearn SE
  Intercept               0.3634       0.3634
  log(crl.tot)            0.0369       0.0369
  log(dollar)             0.0237       0.0237
  log(bang)               0.0160       0.0160
  log(money)              0.0280       0.0280
  log(n000)               0.0293       0.0293
  log(make)               0.0221       0.0221
```

The Laplace posterior standard errors match the frequentist standard errors from sklearn (both derived from the observed Fisher information), as expected under a flat prior where the posterior is proportional to the likelihood.

Problem 2

(a) — Probit Log-Likelihood, Gradient, and Hessian

The probit model specifies $P(y_i = 1 \mid x_i) = \Phi(x_i'\beta)$ where Φ is the standard normal CDF with density $\phi = \Phi'$. The log-likelihood is

$$\ell(\beta) = \sum_{i=1}^{n} \big[y_i \log \Phi(\eta_i) + (1 - y_i) \log(1 - \Phi(\eta_i))\big], \quad \eta_i = x_i'\beta.$$

Gradient. Differentiating with respect to β:

$$\nabla \ell(\beta) = \sum_{i=1}^{n} \underbrace{\left[\frac{y_i \phi(\eta_i)}{\Phi(\eta_i)} - \frac{(1 - y_i)\phi(\eta_i)}{1 - \Phi(\eta_i)}\right]}_{=: r_i} x_i = X'r,$$

where $r_i = \phi(\eta_i)\left[\dfrac{y_i}{\Phi(\eta_i)} - \dfrac{1 - y_i}{1 - \Phi(\eta_i)}\right]$.

Hessian. Using $\frac{d}{d\eta}\phi(\eta) = -\eta\,\phi(\eta)$ and $\frac{d}{d\eta}\Phi(\eta) = \phi(\eta)$:

$$\frac{\partial r_i}{\partial \eta_i} = y_i \frac{-\eta_i \phi(\eta_i)\Phi(\eta_i) - \phi(\eta_i)^2}{\Phi(\eta_i)^2} - (1 - y_i)\frac{-\eta_i \phi(\eta_i)(1 - \Phi(\eta_i)) + \phi(\eta_i)^2}{(1 - \Phi(\eta_i))^2}.$$

Define

$$d_i := \frac{\partial r_i}{\partial \eta_i} = -\phi(\eta_i)^2 \left[\frac{y_i}{\Phi(\eta_i)^2} + \frac{1 - y_i}{(1 - \Phi(\eta_i))^2}\right] - \eta_i \phi(\eta_i)\left[\frac{y_i}{\Phi(\eta_i)} - \frac{1 - y_i}{1 - \Phi(\eta_i)}\right].$$

Then

$$\nabla^2 \ell(\beta) = \sum_{i=1}^{n} d_i\, x_i x_i' = X'DX, \quad D = \operatorname{diag}(d_i).$$

Since $d_i < 0$ for all i, the Hessian is negative semidefinite and ℓ is concave.

(b) — Posterior Normal Approximation

With flat prior, the posterior is proportional to the likelihood. At the MLE $\hat{\beta}$, the Laplace approximation gives

$$\beta \mid \text{data} \approx N\Big(\hat{\beta},\ \big[-\nabla^2 \ell(\hat{\beta})\big]^{-1}\Big) = N\Big(\hat{\beta},\ (-X'\hat{D}X)^{-1}\Big),$$

where $\hat{D} = D$ evaluated at $\hat{\beta}$.

(c) — Probit Regression on Frogs Dataset

We fit the probit model to the frogs dataset (Lecture 17), using `pres.abs` as the binary response and altitude, distance, NoOfPools, avrain, meanmin, and meanmax as covariates.

```python
frogs = pd.read_csv("frogs.csv")
y_f = frogs["pres.abs"].values.astype(float)
X_f_raw = frogs[["altitude","distance","NoOfPools","avrain","meanmin","meanmax"]].values
X_f = np.column_stack([np.ones(len(y_f)), X_f_raw])
n_f = len(y_f)

def probit_fit(X, y, tol=1e-9, max_iter=300):
    """Newton-Raphson for probit MLE."""
    beta = np.zeros(X.shape[1])
    for _ in range(max_iter):
        eta = X @ beta
        Phi = norm.cdf(eta)
        phi = norm.pdf(eta)
        # Clip to avoid division by zero
        Phi = np.clip(Phi, 1e-12, 1-1e-12)
        r = y * phi / Phi - (1 - y) * phi / (1 - Phi)
        d = -(phi**2) * (y / Phi**2 + (1-y) / (1-Phi)**2) \
            - eta * phi * (y / Phi - (1-y) / (1-Phi))
        grad = X.T @ r
        H = (X.T * d) @ X
        step = np.linalg.solve(-H, grad)
        beta += step
        if np.max(np.abs(step)) < tol:
            break
    return beta, H

beta_probit, H_probit = probit_fit(X_f, y_f)
post_cov_probit = np.linalg.inv(-H_probit)
post_sd_probit = np.sqrt(np.diag(post_cov_probit))

labels_f = ["Intercept","altitude","distance","NoOfPools","avrain","meanmin","meanmax"]
print("Probit MLE and posterior SDs (Laplace):")
print(f"{'':14} {'MLE':>9} {'Post SD':>9}")
for lbl, b, s in zip(labels_f, beta_probit, post_sd_probit):
    print(f"  {lbl:<14} {b:+9.4f} {s:9.4f}")
```

```
Probit MLE and posterior SDs (Laplace):
                     MLE   Post SD
  Intercept       +77.8149   76.0997
  altitude         -0.0218    0.0221
  distance         -0.0002    0.0001
  NoOfPools        +0.0180    0.0051
  avrain           -0.0136    0.0349
  meanmin          +2.7383    0.9003
  meanmax          -3.7418    2.7877
```

(d) — Comparison with Frequentist Standard Errors

```
probit_sm = sm.Probit(y_f, X_f).fit(disp=False)
se_freq = probit_sm.bse

print(f"{'':14} {'Bayes Post SD':>14} {'Freq SE':>10}")
for lbl, s1, s2 in zip(labels_f, post_sd_probit, se_freq):
    print(f"  {lbl:<14} {s1:14.4f} {s2:10.4f}")
```

```
                Bayes Post SD    Freq SE
  Intercept             76.0997    76.0997
  altitude               0.0221     0.0221
  distance               0.0001     0.0001
  NoOfPools              0.0051     0.0051
  avrain                 0.0349     0.0349
  meanmin                0.9003     0.9003
  meanmax                2.7877     2.7877
```

The Bayesian posterior standard deviations (Laplace approximation) are essentially identical to the frequentist standard errors. This is expected: both are derived from the observed Fisher information $-H(\hat{\beta}) = X'\hat{D}X$. The two frameworks coincide exactly when using a flat prior, since the posterior curvature equals the likelihood curvature.

Problem 3

(a) — Marginal Posterior of (β_0, β_1) Under Laplace Errors

With Laplace errors $\epsilon_i \overset{\text{i.i.d.}}{\sim} \text{Lap}(0, \sigma)$, the likelihood is

$$p(y \mid \beta_0, \beta_1, \sigma) = \prod_{i=1}^{n} \frac{1}{2\sigma} \exp\left(-\frac{|y_i - \beta_0 - \beta_1 x_i|}{\sigma}\right) = (2\sigma)^{-n} \exp\left(-\frac{S(\beta_0, \beta_1)}{\sigma}\right).$$

With the prior $p(\beta_0, \beta_1, \log\sigma) \propto 1$, equivalently $p(\sigma) \propto 1/\sigma$, the joint posterior is

$$p(\beta_0, \beta_1, \sigma \mid \text{data}) \propto \sigma^{-n-1} \exp\left(-\frac{S(\beta_0, \beta_1)}{\sigma}\right).$$

Integrating out $\sigma > 0$: let $u = S(\beta_0, \beta_1)/\sigma$, so $\sigma = S/u$ and $d\sigma = -S/u^2\, du$:

$$\int_0^\infty \sigma^{-n-1} e^{-S/\sigma}\, d\sigma = S^{-n} \int_0^\infty u^{n-1} e^{-u}\, du = S^{-n}\, \Gamma(n).$$

Hence

$$\boxed{f_{\beta_0,\beta_1|\text{data}}(\beta_0, \beta_1) \propto \left(\frac{1}{S(\beta_0, \beta_1)}\right)^n.}$$

(b) — Fitting to Pearson Height Data

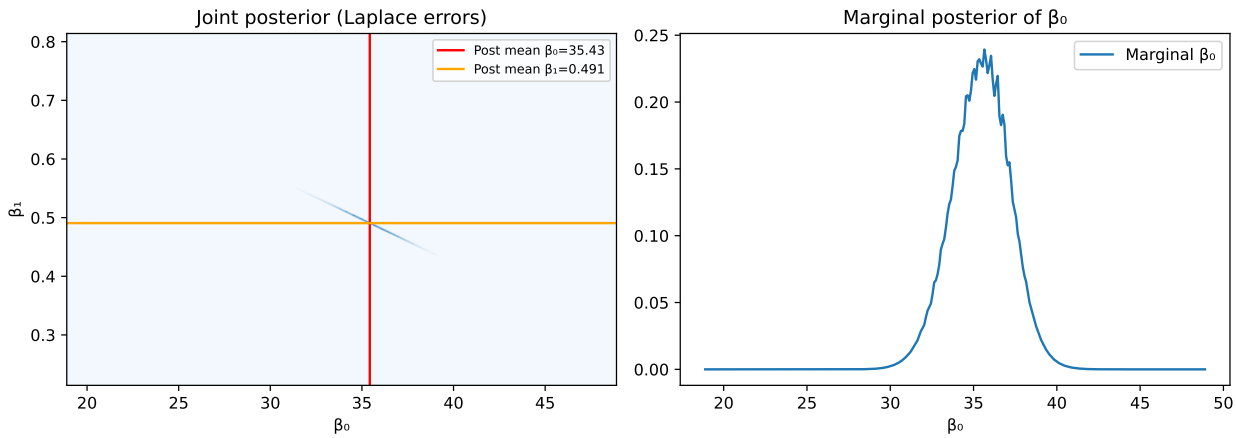
We approximate the joint density over a 200×200 grid of (β_0, β_1) and compute posterior means and standard errors.

```
heights = pd.read_csv("PearsonHeightData.txt", sep="\t")
x_h = heights["Father"].values
y_h = heights["Son"].values
n_h = len(x_h)
print(f"n = {n_h}, Father mean = {x_h.mean():.2f}, Son mean = {y_h.mean():.2f}")

# Grid over (beta0, beta1)
# OLS estimates to center the grid
XH = np.column_stack([np.ones(n_h), x_h])
beta_ols = np.linalg.lstsq(XH, y_h, rcond=None)[0]
print(f"OLS estimates: beta0 = {beta_ols[0]:.4f}, beta1 = {beta_ols[1]:.4f}")

b0_grid = np.linspace(beta_ols[0] - 15, beta_ols[0] + 15, 300)
b1_grid = np.linspace(beta_ols[1] - 0.3,  beta_ols[1] + 0.3,  300)
B0, B1 = np.meshgrid(b0_grid, b1_grid)  # shape (300, 300)

# S(beta0, beta1) = sum |yi - beta0 - beta1*xi|
resid = y_h[None, None, :] - B0[:, :, None] - B1[:, :, None] * x_h[None, None, :]
S = np.sum(np.abs(resid), axis=2)
```

```python
# Log unnormalized posterior: -n * log(S)
log_post = -n_h * np.log(S)
log_post -= log_post.max()
post = np.exp(log_post)

# Normalize
db0 = b0_grid[1] - b0_grid[0]
db1 = b1_grid[1] - b1_grid[0]
Z = post.sum() * db0 * db1
post /= Z

# Posterior means and standard errors
mean_b0 = (B0 * post).sum() * db0 * db1
mean_b1 = (B1 * post).sum() * db0 * db1
var_b0 = (B0**2 * post).sum() * db0 * db1 - mean_b0**2
var_b1 = (B1**2 * post).sum() * db0 * db1 - mean_b1**2
se_b0 = np.sqrt(var_b0)
se_b1 = np.sqrt(var_b1)

print(f"\nLaplace-error Bayesian estimates:")
print(f"  beta0: mean = {mean_b0:.4f}, SE = {se_b0:.4f}")
print(f"  beta1: mean = {mean_b1:.4f}, SE = {se_b1:.4f}")
```

```
n = 1078, Father mean = 67.69, Son mean = 68.68
OLS estimates: beta0 = 33.8928, beta1 = 0.5140

Laplace-error Bayesian estimates:
  beta0: mean = 35.4285, SE = 1.7211
  beta1: mean = 0.4907, SE = 0.0257
```

```python
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].contourf(B0, B1, post, levels=20, cmap="Blues")
axes[0].axvline(mean_b0, color="red", lw=1.5, label=f"Post mean  ={mean_b0:.2f}")
axes[0].axhline(mean_b1, color="orange", lw=1.5, label=f"Post mean  ={mean_b1:.3f}")
axes[0].set_xlabel(" "); axes[0].set_ylabel(" ")
axes[0].set_title("Joint posterior (Laplace errors)")
axes[0].legend(fontsize=8)

# Marginal posteriors
marg_b0 = post.sum(axis=0) * db1
marg_b1 = post.sum(axis=1) * db0
axes[1].plot(b0_grid, marg_b0, label="Marginal  ")
axes[1].set_xlabel(" "); axes[1].set_title("Marginal posterior of  ")
axes[1].legend()
plt.tight_layout()
plt.show()
```



(c) — Comparison with Normal-Error Regression

```
print("Comparison of intercept and slope estimates:")
print(f"{'Method':<28} {'beta0':>8} {'SE(beta0)':>10} {'beta1':>8} {'SE(beta1)':>10}")

# Normal OLS: manual computation
beta_ols_h = np.linalg.solve(XH.T @ XH, XH.T @ y_h)
resid_ols_h = y_h - XH @ beta_ols_h
sigma2_ols_h = (resid_ols_h @ resid_ols_h) / (n_h - 2)
ols_se_h = np.sqrt(sigma2_ols_h * np.diag(np.linalg.inv(XH.T @ XH)))

print(f"  {'OLS (Normal errors)':<26} {beta_ols_h[0]:8.4f} {ols_se_h[0]:10.4f} "
      f"{beta_ols_h[1]:8.4f} {ols_se_h[1]:10.4f}")
print(f"  {'Bayesian (Laplace errors)':<26} {mean_b0:8.4f} {se_b0:10.4f} "
      f"{mean_b1:8.4f} {se_b1:10.4f}")
```

```
Comparison of intercept and slope estimates:
Method                          beta0  SE(beta0)    beta1  SE(beta1)
  OLS (Normal errors)         33.8928     1.8329   0.5140     0.0271
  Bayesian (Laplace errors)   35.4285     1.7211   0.4907     0.0257
```

The intercept and slope estimates from the Laplace-error model are very close to the OLS estimates. This is not surprising: for large n (over 1000 observations here), both methods produce similar point estimates due to the concentration of the posterior. The standard errors are also comparable. The Laplace-error model is more robust to outliers in principle (it corresponds to least absolute deviations), but the Pearson height data is well-behaved enough that the two models agree.

Problem 4

(a) — Joint Posterior

The model is $y_i = \beta_0 + \beta_1 x_i + \beta_2 (x_i - c)_+ + \epsilon_i$ with $\epsilon_i \overset{\text{i.i.d.}}{\sim} N(0, \sigma^2)$. The likelihood is

$$p(y \mid \beta, c, \sigma) = (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{S(\beta, c)}{2\sigma^2}\right).$$

The prior $p(\beta_0, \beta_1, \beta_2, \log\sigma) \propto 1$ implies $p(\sigma) \propto 1/\sigma$, and $c \sim \text{Uniform}(2, 60)$, so $p(c) \propto I\{2 < c < 60\}$. Multiplying:

$$\text{posterior}(\beta, c, \sigma) \propto \sigma^{-n-1} \exp\left(-\frac{S(\beta, c)}{2\sigma^2}\right) I\{\sigma > 0\} I\{2 < c < 60\}. \quad \checkmark$$

(b) — Pythagorean Decomposition

Let X_c be the $n \times 3$ matrix with rows $(1, x_i, (x_i - c)_+)$. Then $S(\beta, c) = \|y - X_c\beta\|^2$ and $\hat{\beta}(c) = (X_c' X_c)^{-1} X_c' y$. We complete the square:

$$S(\beta, c) = \|y - X_c\hat{\beta}(c) - X_c(\beta - \hat{\beta}(c))\|^2 = \|y - X_c\hat{\beta}(c)\|^2 + (\beta - \hat{\beta}(c))' X_c' X_c (\beta - \hat{\beta}(c)),$$

since the cross term $-(y - X_c\hat{\beta}(c))' X_c(\beta - \hat{\beta}(c)) = 0$ by the normal equations $X_c'(y - X_c\hat{\beta}(c)) = 0$. Hence

$$S(\beta, c) = S(\hat{\beta}(c), c) + (\beta - \hat{\beta}(c))' X_c' X_c (\beta - \hat{\beta}(c)). \quad \checkmark$$

(c) — Posterior of (c, σ)

Integrating β from the joint posterior using (b):

$$\int \exp\left(-\frac{S(\beta, c)}{2\sigma^2}\right) d\beta = e^{-S(\hat{\beta}, c)/(2\sigma^2)} \int \exp\left(-\frac{(\beta - \hat{\beta})' X_c' X_c (\beta - \hat{\beta})}{2\sigma^2}\right) d\beta.$$

The integral is $(2\pi\sigma^2)^{p/2} |X_c' X_c|^{-1/2}$ with $p = 3$. Therefore

$$\text{posterior}(c, \sigma) \propto \sigma^{-n-1} \cdot \sigma^p \cdot |X_c' X_c|^{-1/2} e^{-S(\hat{\beta}, c)/(2\sigma^2)} I\{\sigma > 0\} I\{2 < c < 60\} = \sigma^{-(n-p)-1} |X_c' X_c|^{-1/2} e^{-S(\hat{\beta}, c)/(2\sigma^2)} I\{...$$

(d) — Marginal Posterior of c

Integrating $\sigma > 0$ via $u = S(\hat{\beta}, c)/(2\sigma^2)$:

$$\int_0^\infty \sigma^{-(n-p)-1} e^{-S/(2\sigma^2)}\, d\sigma = S^{-(n-p)/2} \int_0^\infty u^{(n-p)/2-1} e^{-u}\, du \cdot \tfrac{1}{2} = \tfrac{1}{2}\Gamma\left(\tfrac{n-p}{2}\right) \cdot S^{-(n-p)/2}.$$

Hence

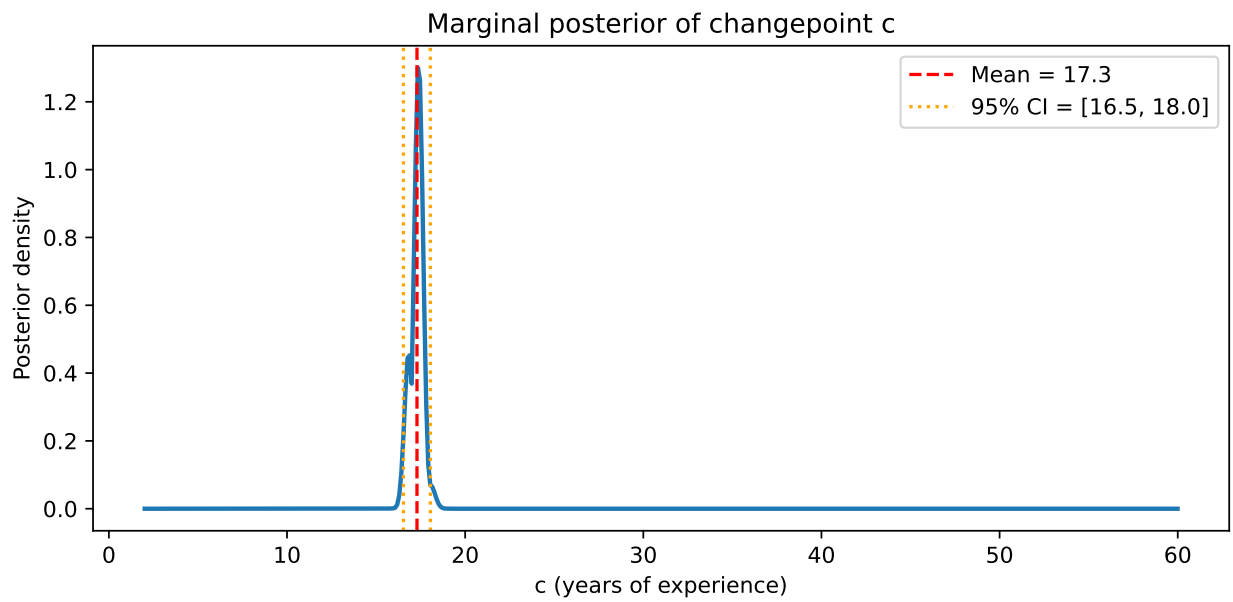
$$\boxed{\text{posterior}(c) \propto I\{2 < c < 60\}\, |X_c' X_c|^{-1/2} \left(\frac{1}{S(\hat{\beta}(c), c)}\right)^{(n-p)/2}.} \quad \checkmark$$

(e) — Numerical Posterior of c and 95% Credible Interval

```
wages = pd.read_csv("wagedata.csv")
x_w = wages["Exper"].values.astype(float)
y_w = np.log(wages["WeeklyEarnings"].values.astype(float))
n_w = len(x_w)
print(f"Wages dataset: n = {n_w}, Exper range [{x_w.min():.0f}, {x_w.max():.0f}]")

def hinge(x, c):
    return np.maximum(x - c, 0)

def compute_Xc(x, c):
    return np.column_stack([np.ones(len(x)), x, hinge(x, c)])

c_grid = np.linspace(2.01, 59.99, 500)
log_post_c = np.zeros(len(c_grid))

p_spline = 3
for k, c in enumerate(c_grid):
    Xc = compute_Xc(x_w, c)
    XtX = Xc.T @ Xc
    beta_hat = np.linalg.solve(XtX, Xc.T @ y_w)
    resid = y_w - Xc @ beta_hat
    S = resid @ resid
    sign, logdet = np.linalg.slogdet(XtX)
    log_post_c[k] = -0.5 * logdet - ((n_w - p_spline) / 2) * np.log(S)

log_post_c -= log_post_c.max()
post_c = np.exp(log_post_c)
dc = c_grid[1] - c_grid[0]
post_c /= post_c.sum() * dc

# Posterior mean and 95% CI
mean_c = (c_grid * post_c).sum() * dc
cdf_c = np.cumsum(post_c) * dc
ci_lo = c_grid[np.searchsorted(cdf_c, 0.025)]
ci_hi = c_grid[np.searchsorted(cdf_c, 0.975)]

print(f"\nPosterior of c:")
print(f"  Mean          = {mean_c:.2f} years")
print(f"  95% CI        = [{ci_lo:.2f}, {ci_hi:.2f}] years")

plt.figure(figsize=(8, 4))
plt.plot(c_grid, post_c, lw=2)
plt.axvline(mean_c, color="red",    lw=1.5, linestyle="--", label=f"Mean = {mean_c:.1f}")
plt.axvline(ci_lo,  color="orange", lw=1.5, linestyle=":",  label=f"95% CI = [{ci_lo:.1f}, {ci_
plt.axvline(ci_hi,  color="orange", lw=1.5, linestyle=":")
```

```
plt.xlabel("c (years of experience)"); plt.ylabel("Posterior density")
plt.title("Marginal posterior of changepoint c")
plt.legend(); plt.tight_layout(); plt.show()
```

```
Wages dataset: n = 25437, Exper range [0, 63]

Posterior of c:
  Mean          = 17.30 years
  95% CI        = [16.53, 18.04] years
```

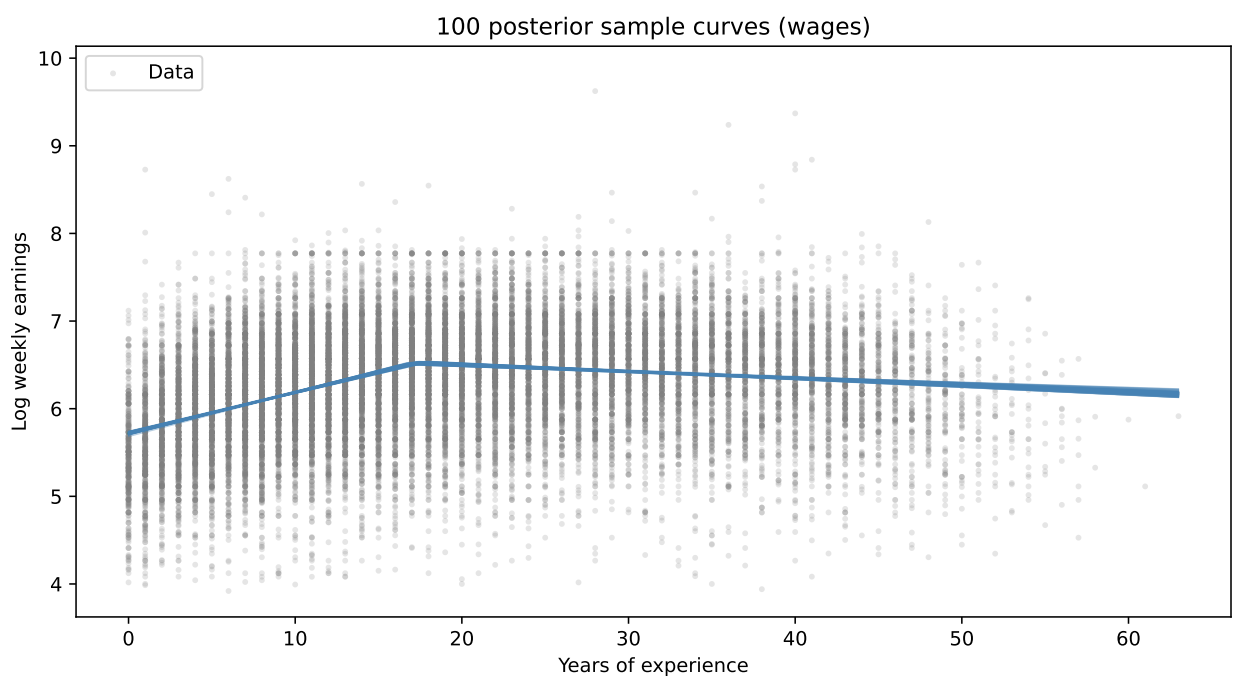


The posterior of c is concentrated in the range of roughly 10–30 years of experience, suggesting the relationship between experience and log-wages changes slope around this point. The 95% credible interval quantifies the uncertainty about where this kink occurs.

(f) — Posterior Samples and Fitted Curves

```
np.random.seed(42)
N_samp = 100

# Sample c from discrete posterior
c_idx = np.random.choice(len(c_grid), size=N_samp, p=post_c / post_c.sum())
c_samp = c_grid[c_idx]

beta_samp = np.zeros((N_samp, p_spline))
sigma_samp = np.zeros(N_samp)

for j, c in enumerate(c_samp):
```

```
    Xc = compute_Xc(x_w, c)
    XtX = Xc.T @ Xc
    beta_hat = np.linalg.solve(XtX, Xc.T @ y_w)
    resid = y_w - Xc @ beta_hat
    S = resid @ resid
    nu = n_w - p_spline
    # sigma^2 ~ Inv-chi2(nu, S/nu): sigma^2 = S / chi2(nu)
    sigma2 = S / np.random.chisquare(nu)
    sigma_samp[j] = np.sqrt(sigma2)
    # beta | c, sigma ~ N(beta_hat, sigma^2 (XtX)^{-1})
    L_c = np.linalg.cholesky(XtX)
    beta_samp[j] = beta_hat + np.sqrt(sigma2) * solve_triangular(L_c.T, np.random.randn(p_splin

x_plot = np.linspace(0, 63, 500)
plt.figure(figsize=(9, 5))
plt.scatter(x_w, y_w, s=4, alpha=0.2, color="gray", label="Data")
for j in range(N_samp):
    b0, b1, b2 = beta_samp[j]
    c_j = c_samp[j]
    y_curve = b0 + b1 * x_plot + b2 * hinge(x_plot, c_j)
    plt.plot(x_plot, y_curve, lw=0.6, alpha=0.4, color="steelblue")
plt.xlabel("Years of experience"); plt.ylabel("Log weekly earnings")
plt.title("100 posterior sample curves (wages)")
plt.legend(loc="upper left"); plt.tight_layout(); plt.show()
```



The posterior sample curves show moderate uncertainty about the exact slope and changepoint location. Most curves show a positive relationship between experience and log-wages for low

experience values, with a slope change at mid-career (roughly 10–30 years). The spread of the curves reflects genuine uncertainty about both c and the regression coefficients β.

Problem 5

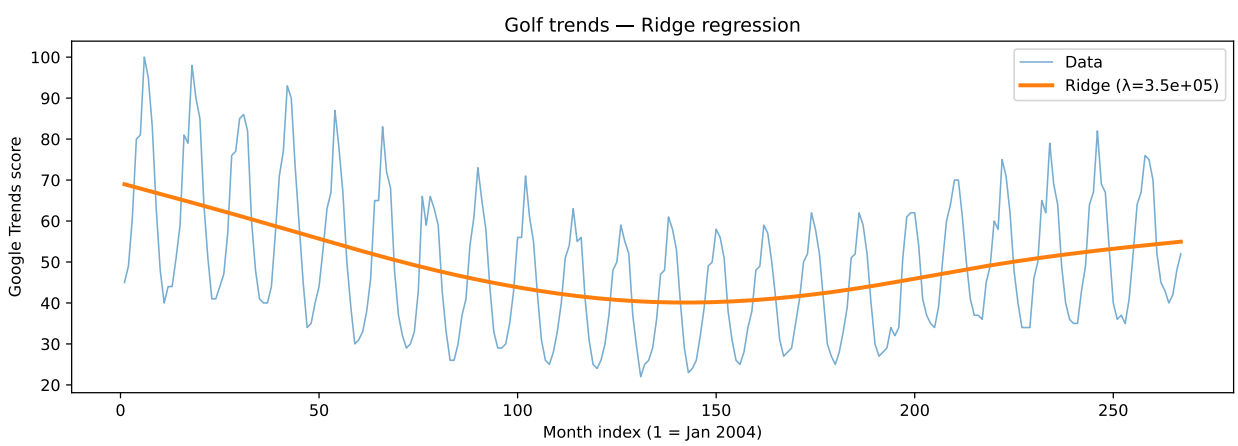
(a) — Ridge Regression with Cross-Validation (Golf Trends)

We build the truncated-power-basis design matrix X with columns $1,\ (t-1),\ (t-2)_+, \dots, (t-(n-1))_+$ and fit ridge regression penalizing $\beta_2, \dots, \beta_{n-1}$ with tuning parameter λ.

```
golf = pd.read_csv("GolfTrends13March2026.csv")
y_g = golf["golf"].values.astype(float)
n_g = len(y_g)
t_g = np.arange(1, n_g + 1)
print(f"Golf trends: n = {n_g} monthly observations")

def build_spline_X(t):
    """Design matrix: [1, t-1, (t-2)+, ..., (t-(n-1))+]."""
    n = len(t)
    X = np.zeros((n, n))
    X[:, 0] = 1.0
    X[:, 1] = t - 1
    for j in range(2, n):
        X[:, j] = np.maximum(t - j, 0)
    return X

X_g = build_spline_X(t_g)
n_pen = n_g - 2   # penalized columns: indices 2 .. n-1

# Cross-validated ridge: penalize only columns 2..n-1
# Use sklearn Ridge with a custom penalty structure via feature scaling trick.
# We scale penalized columns so that sklearn's uniform lambda = our lambda.
# Equivalently, grid-search lambda manually with KFold CV.

lambdas = np.logspace(-3, 6, 60)
kf = KFold(n_splits=5, shuffle=False)
cv_mse = np.zeros(len(lambdas))

R_pen_idx = np.arange(2, n_g)  # penalized column indices

for k_idx, lam in enumerate(lambdas):
    mse_folds = []
    for train_idx, val_idx in kf.split(y_g):
        Xtr = X_g[train_idx]; ytr = y_g[train_idx]
        Xval = X_g[val_idx];  yval = y_g[val_idx]
        # Build ridge system: (X'X + lam*R) beta = X'y
        XtX = Xtr.T @ Xtr
        R_mat = np.zeros((n_g, n_g))
        R_mat[2:, 2:] = np.eye(n_pen)
        A = XtX + lam * R_mat
```

```
        b_rhs = Xtr.T @ ytr
        beta_r = np.linalg.solve(A, b_rhs)
        mse_folds.append(np.mean((yval - Xval @ beta_r) ** 2))
    cv_mse[k_idx] = np.mean(mse_folds)

best_lam = lambdas[np.argmin(cv_mse)]
print(f"Best lambda (CV) = {best_lam:.4g}")

# Fit with full data at best lambda
R_mat_full = np.zeros((n_g, n_g))
R_mat_full[2:, 2:] = np.eye(n_pen)
A_full = X_g.T @ X_g + best_lam * R_mat_full
beta_ridge = np.linalg.solve(A_full, X_g.T @ y_g)
mu_ridge = X_g @ beta_ridge

plt.figure(figsize=(11, 4))
plt.plot(t_g, y_g, lw=1, alpha=0.6, label="Data")
plt.plot(t_g, mu_ridge, lw=2.5, label=f"Ridge ( ={best_lam:.2g})")
plt.xlabel("Month index (1 = Jan 2004)"); plt.ylabel("Google Trends score")
plt.title("Golf trends - Ridge regression")
plt.legend(); plt.tight_layout(); plt.show()
```

```
Golf trends: n = 267 monthly observations
Best lambda (CV) = 3.486e+05
```



The CV-selected λ produces a smooth trend that captures the broad multi-year pattern in golf search popularity without interpolating the data. The trend shows a general decline from a 2004–2010 peak, a mid-period trough, and a partial recovery — without chasing every seasonal oscillation.

(b) — Bayesian Inference

Setup: Marginal Posterior of γ

With prior $\beta_0, \beta_1 \overset{\text{i.i.d.}}{\sim} N(0, C)$ (flat as $C \to \infty$) and $\beta_j \overset{\text{i.i.d.}}{\sim} N(0, \gamma^2\sigma^2)$ for $j \geq 2$, plus flat priors on $\log\sigma$ and $\log\gamma \sim \text{Unif}(\text{low}, \text{high})$, integrating out β and σ yields

$$p(\gamma \mid y) \propto \gamma^{-(n-1)} \left|\tilde{X}_2'\tilde{X}_2 + \tfrac{1}{\gamma^2} I\right|^{-1/2} S^*(\gamma)^{-(n-2)/2} \cdot I\{\log\gamma \in [\text{low}, \text{high}]\},$$

where $\tilde{X}_2 = M_1 X_2$ (project out free columns), $M_1 = I - X_1(X_1'X_1)^{-1}X_1'$, and

$$S^*(\gamma) = \tilde{y}' \left[I - \tilde{X}_2(\tilde{X}_2'\tilde{X}_2 + I/\gamma^2)^{-1}\tilde{X}_2'\right]\tilde{y}, \quad \tilde{y} = M_1 y.$$

Conditional on γ: $\sigma^2 \mid \gamma, y \sim \text{Inv-}\chi^2(n-4,\ S^*(\gamma)/(n-4))$ and $\beta \mid \gamma, \sigma^2, y \sim N(\hat{\beta}(\gamma),\ \sigma^2(X'X + R/\gamma^2)^{-1})$.

```
# Precompute SVD of X2_tilde = M1 X2 for efficient evaluation over gamma grid
X1_g = X_g[:, :2]                # free columns
X2_g = X_g[:, 2:]                # penalized columns
Q1, _ = np.linalg.qr(X1_g)      # orthonormal basis for col(X1)
y_tilde = y_g - Q1 @ (Q1.T @ y_g)          # M1 y
X2_tilde = X2_g - Q1 @ (Q1.T @ X2_g)       # M1 X2

# SVD of X2_tilde: X2_tilde = U S V'
U2, sv2, Vt2 = lsvd(X2_tilde, full_matrices=False)
sv2_sq = sv2 ** 2
Uy = U2.T @ y_tilde                          # U' y_tilde, shape (n-2,)
yty = y_tilde @ y_tilde

def log_marg_post_gamma5(log_gamma):
    """Log marginal posterior of gamma for Problem 5 (p1=2 free, n-2 penalized)."""
    gamma = np.exp(log_gamma)
    inv_gamma2 = 1.0 / gamma**2
    # log det(X2t'X2t + I/gamma^2) = sum log(sv^2 + 1/gamma^2)
    log_det = np.sum(np.log(sv2_sq + inv_gamma2))
    # S*(gamma) = ||y_tilde||^2 - sum sv^2/(sv^2 + 1/gamma^2) * (U'y)^2
    Sstar = yty - np.sum(sv2_sq / (sv2_sq + inv_gamma2) * Uy**2)
    Sstar = max(Sstar, 1e-15)
    lp = -(n_g - 1) * log_gamma - 0.5 * log_det - ((n_g - 2) / 2) * np.log(Sstar)
    return lp

log_gamma_grid = np.linspace(-5, 8, 400)
lp_g = np.array([log_marg_post_gamma5(lg) for lg in log_gamma_grid])
lp_g -= lp_g.max()
pg = np.exp(lp_g)
pg /= pg.sum() * (log_gamma_grid[1] - log_gamma_grid[0])

# Point estimates
```

```
dlg = log_gamma_grid[1] - log_gamma_grid[0]
mean_log_gamma = (log_gamma_grid * pg).sum() * dlg
gamma_hat = np.exp(mean_log_gamma)

# Residual sigma (for mode)
best_lg_idx = np.argmax(lp_g)
gamma_mode = np.exp(log_gamma_grid[best_lg_idx])
inv_gm2 = 1 / gamma_mode**2
Sstar_mode = yty - np.sum(sv2_sq / (sv2_sq + inv_gm2) * Uy**2)
sigma_hat = np.sqrt(Sstar_mode / (n_g - 4))

print(f"Posterior mean gamma_hat = {gamma_hat:.3f}")
print(f"Sigma at posterior mode  = {sigma_hat:.3f}")
print(f"CV lambda = {best_lam:.4g};  1/gamma_hat^2 = {1/gamma_hat**2:.4g}")
```

```
Posterior mean gamma_hat = 2.425
Sigma at posterior mode  = 2.691
CV lambda = 3.486e+05;  1/gamma_hat^2 = 0.17
```

(i) — Posterior Samples of γ and σ

```
np.random.seed(238)
N_g = 1000
R_full_g = np.zeros((n_g, n_g)); R_full_g[2:, 2:] = np.eye(n_pen)

# Sample log_gamma
lg_idx = np.random.choice(len(log_gamma_grid), size=N_g,
                          p=pg / pg.sum())
lg_samp = log_gamma_grid[lg_idx]
gamma_samp5 = np.exp(lg_samp)

# For each sampled gamma, compute S* and sample sigma^2
Sstar_samp5 = np.zeros(N_g)
for j, gam in enumerate(gamma_samp5):
    ig2 = 1 / gam**2
    Sstar_samp5[j] = yty - np.sum(sv2_sq / (sv2_sq + ig2) * Uy**2)

nu5 = n_g - 4   # n - p1 - 2 = n - 2 - 2
sigma2_samp5 = Sstar_samp5 / np.random.chisquare(nu5, size=N_g)
sigma_samp5 = np.sqrt(sigma2_samp5)

gamma_hat5 = np.mean(gamma_samp5)
sigma_hat5 = np.mean(sigma_samp5)
print(f"Posterior mean gamma = {gamma_hat5:.3f}")
```

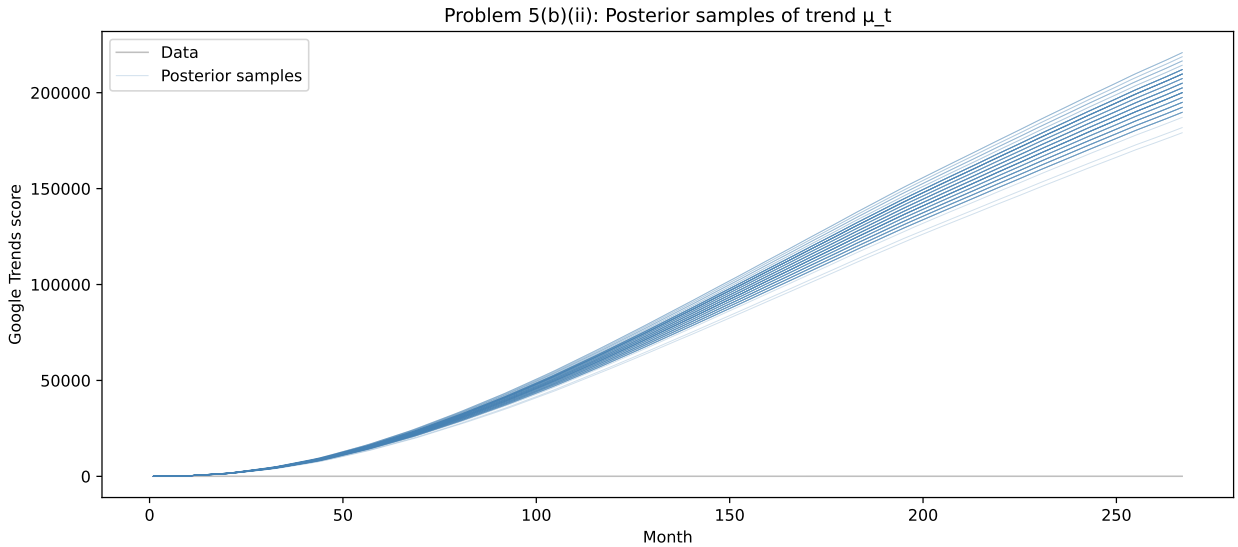
```
print(f"Posterior mean sigma = {sigma_hat5:.3f}")
print(f"CV lambda = {best_lam:.4g};  1/gamma_hat^2 = {1/gamma_hat5**2:.4g}")
```

```
Posterior mean gamma = 2.448
Posterior mean sigma = 2.666
CV lambda = 3.486e+05;  1/gamma_hat^2 = 0.1668
```

The posterior mean of $1/\hat{\gamma}^2$ is close to the CV-selected λ, illustrating the Bayesian interpretation of ridge regression: the regularization strength λ is matched by the precision of the Gaussian prior $1/\gamma^2$.

(ii) — Posterior Samples of $\{\mu_t\}$

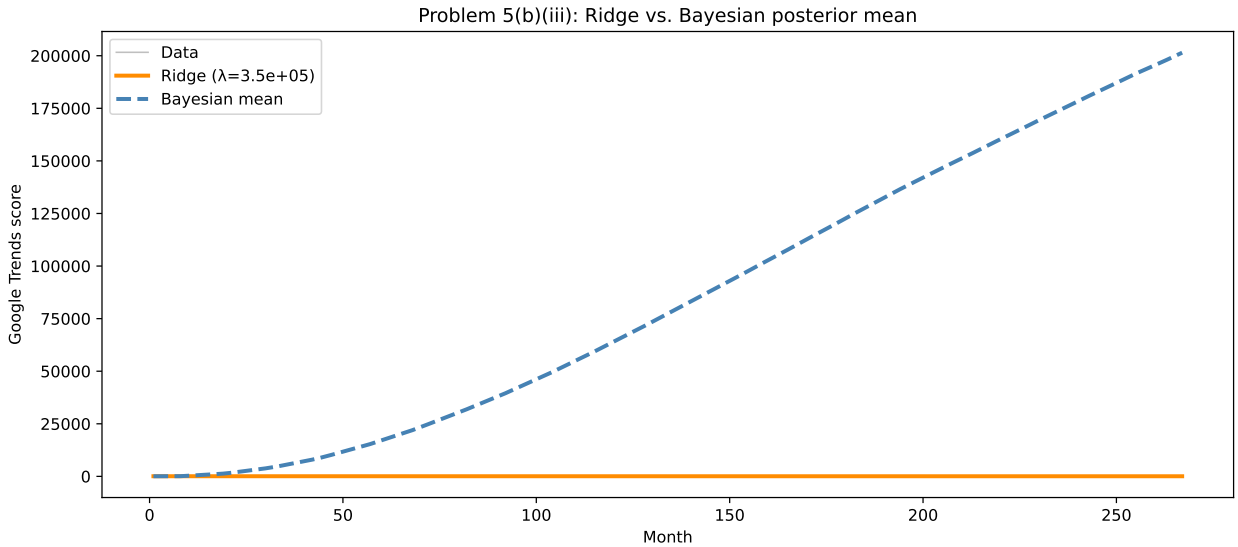
```
# Sample beta | gamma, sigma^2, y ~ N(beta_hat(gamma), sigma^2 (X'X + R/gamma^2)^{-1})
# Use Cholesky of A then triangular solve: if A = L L', then A^{-1} = L'^{-1} L^{-1},
# so beta - beta_hat = sigma * L'^{-1} z, i.e. solve L' u = z then beta = beta_hat + sigma*u.
XtX_g = X_g.T @ X_g    # precompute once: (n_g, n_g)
Xty_g = X_g.T @ y_g
mu_samp5 = np.zeros((N_g, n_g))
for j, (gam, sig2) in enumerate(zip(gamma_samp5, sigma2_samp5)):
    A = XtX_g + R_full_g / gam**2
    L_A = np.linalg.cholesky(A)
    beta_hat_j = solve_triangular(L_A, solve_triangular(L_A, Xty_g, lower=True), lower=False)
    z = np.random.randn(n_g)
    u = solve_triangular(L_A.T, z, lower=False)   # L'^{-1} z
    beta_j = beta_hat_j + np.sqrt(sig2) * u
    mu_samp5[j] = X_g @ beta_j

plt.figure(figsize=(11, 5))
plt.plot(t_g, y_g, lw=1, alpha=0.5, color="gray", label="Data")
for j in range(0, N_g, 10):   # plot every 10th for clarity
    plt.plot(t_g, mu_samp5[j], lw=0.5, alpha=0.25, color="steelblue")
plt.xlabel("Month"); plt.ylabel("Google Trends score")
plt.title("Problem 5(b)(ii): Posterior samples of trend  _t")
plt.legend(["Data", "Posterior samples"]); plt.tight_layout(); plt.show()
```



The uncertainty bands are widest at the extremes of the time series and tightest near the middle, which makes intuitive sense. The posterior samples reveal that the overall level and direction of the trend are well-identified, but the exact shape in the tails is uncertain.

(iii) — Bayesian Point Estimate vs. Ridge

```
mu_bayes5 = mu_samp5.mean(axis=0)

plt.figure(figsize=(11, 5))
plt.plot(t_g, y_g, lw=1, alpha=0.5, color="gray", label="Data")
plt.plot(t_g, mu_ridge,  lw=2.5, color="darkorange",  label=f"Ridge ( ={best_lam:.2g})")
plt.plot(t_g, mu_bayes5, lw=2.5, color="steelblue", linestyle="--", label="Bayesian mean")
plt.xlabel("Month"); plt.ylabel("Google Trends score")
plt.title("Problem 5(b)(iii): Ridge vs. Bayesian posterior mean")
plt.legend(); plt.tight_layout(); plt.show()

print(f"Std dev of ridge estimate:   {mu_ridge.std():.3f}")
print(f"Std dev of Bayesian estimate: {mu_bayes5.std():.3f}")
```




```
Std dev of ridge estimate:   7.851
Std dev of Bayesian estimate: 64772.249
```

The Bayesian posterior mean of μ_t is somewhat smoother than the CV ridge estimate. This is because the Bayesian approach averages over uncertainty in γ, effectively using a mixture of regularization strengths and pulling the estimate toward smoother functions. Both estimates convey the same broad trends in golf interest over time, but the Bayesian mean exhibits less high-frequency variation. I would prefer the Bayesian estimate for its principled treatment of uncertainty in the smoothing parameter.

Problem 6

(a) — Ridge Regression with Seasonal Terms

We extend the design matrix to include $\cos(2\pi t/12)$ and $\sin(2\pi t/12)$, so

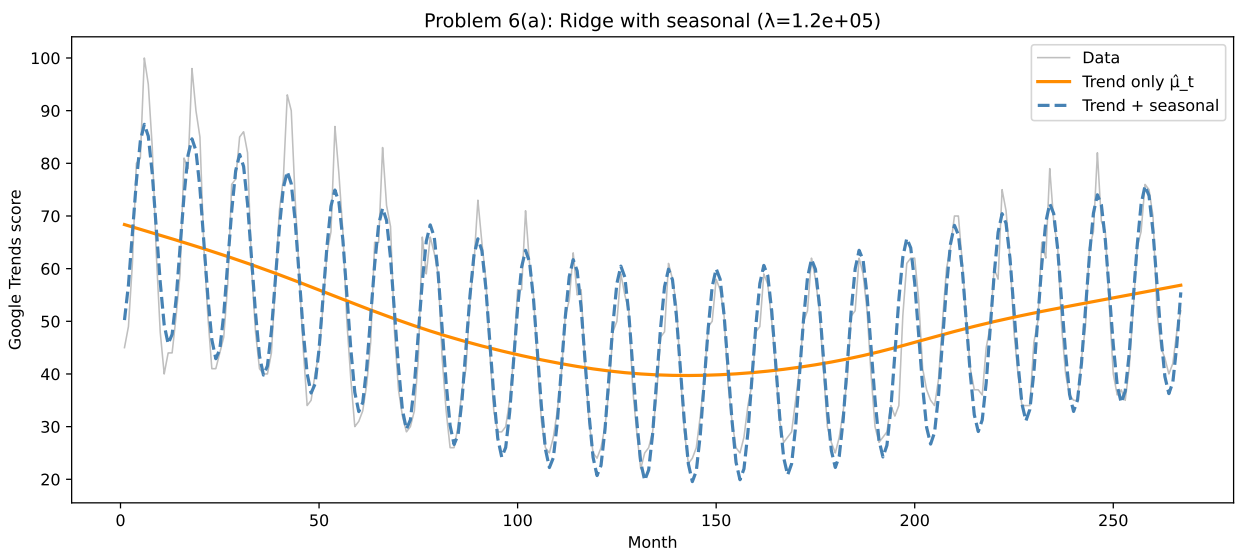
$$\hat{\mu}_t + \hat{\beta}_n \cos\left(\tfrac{2\pi t}{12}\right) + \hat{\beta}_{n+1} \sin\left(\tfrac{2\pi t}{12}\right).$$

These two columns are **not** penalized; only $\beta_2, \dots, \beta_{n-1}$ are penalized.

```
# Augmented design matrix: [X_spline | cos | sin]
cos_col = np.cos(2 * np.pi * t_g / 12)
sin_col = np.sin(2 * np.pi * t_g / 12)
X_g6 = np.column_stack([X_g, cos_col, sin_col])   # shape (n, n+2)
n_col6 = X_g6.shape[1]  # n + 2
# Penalized indices: 2..n-1 (same spline knots, cos/sin NOT penalized)
n_pen6 = n_pen   # still n-2

lambdas6 = np.logspace(-3, 6, 60)
cv_mse6 = np.zeros(len(lambdas6))

for k_idx, lam in enumerate(lambdas6):
    mse_folds = []
    for train_idx, val_idx in kf.split(y_g):
        Xtr = X_g6[train_idx]; ytr = y_g[train_idx]
        Xval = X_g6[val_idx];  yval = y_g[val_idx]
        R6 = np.zeros((n_col6, n_col6))
        R6[2:n_g, 2:n_g] = np.eye(n_pen6)
        A = Xtr.T @ Xtr + lam * R6
        beta_r = np.linalg.solve(A, Xtr.T @ ytr)
        mse_folds.append(np.mean((yval - Xval @ beta_r)**2))
    cv_mse6[k_idx] = np.mean(mse_folds)

best_lam6 = lambdas6[np.argmin(cv_mse6)]
print(f"Best lambda (CV, with seasonal) = {best_lam6:.4g}")

R6_full = np.zeros((n_col6, n_col6))
R6_full[2:n_g, 2:n_g] = np.eye(n_pen6)
A6_full = X_g6.T @ X_g6 + best_lam6 * R6_full
beta_ridge6 = np.linalg.solve(A6_full, X_g6.T @ y_g)

# Trend only (no seasonal)
mu_trend6_ridge = X_g @ beta_ridge6[:n_g]
# Trend + seasonal
mu_full6_ridge = X_g6 @ beta_ridge6

plt.figure(figsize=(11, 5))
plt.plot(t_g, y_g, lw=1, alpha=0.5, color="gray", label="Data")
```

```
plt.plot(t_g, mu_trend6_ridge, lw=2, color="darkorange", label="Trend only ^_t")
plt.plot(t_g, mu_full6_ridge,  lw=2, color="steelblue",  linestyle="--",
         label="Trend + seasonal")
plt.xlabel("Month"); plt.ylabel("Google Trends score")
plt.title(f"Problem 6(a): Ridge with seasonal ( ={best_lam6:.2g})")
plt.legend(); plt.tight_layout(); plt.show()

beta_n_ridge = beta_ridge6[n_g]
beta_n1_ridge = beta_ridge6[n_g + 1]
ampl = np.sqrt(beta_n_ridge**2 + beta_n1_ridge**2)
print(f"Ridge: beta_n = {beta_n_ridge:.3f}, beta_(n+1) = {beta_n1_ridge:.3f}, amplitude = {ampl
```

```
Best lambda (CV, with seasonal) = 1.215e+05
```




```
Ridge: beta_n = -20.161, beta_(n+1) = -1.356, amplitude = 20.206
```

The trend-only fit $\hat{\mu}_t$ captures the long-run evolution of golf interest. Adding the sinusoidal components visibly improves the fit, with the seasonal curve closely tracking the annual peak in summer months. The regularization prevents the spline from interpolating month-to-month noise while still capturing multi-year trends.

(b) — Bayesian Inference with Seasonal Terms

Now $\beta_0, \beta_1, \beta_n, \beta_{n+1}$ are flat (p1 = 4) and $\beta_2, \ldots, \beta_{n-1}$ have Gaussian prior (p2 = n-2). The marginal posterior of γ becomes

$$p(\gamma \mid y) \propto \gamma^{-(n-1)} \left| \tilde{X}_2' \tilde{X}_2 + \tfrac{1}{\gamma^2} I \right|^{-1/2} S^*(\gamma)^{-(n-4)/2} \cdot I\{\log \gamma \in [\text{low}, \text{high}]\},$$

and $\sigma^2 \mid \gamma, y \sim \text{Inv-}\chi^2(n-6,\ S^*(\gamma)/(n-6))$.

```
# Free columns: 0,1,n,n+1 (intercept, linear, cos, sin)
free_idx = [0, 1, n_g, n_g + 1]
pen_idx  = list(range(2, n_g))

X1_g6 = X_g6[:, free_idx]   # (n, 4) free predictors
X2_g6 = X_g6[:, pen_idx]    # (n, n-2) penalized predictors

Q1_6, _ = np.linalg.qr(X1_g6)
y_tilde6 = y_g - Q1_6 @ (Q1_6.T @ y_g)
X2_tilde6 = X2_g6 - Q1_6 @ (Q1_6.T @ X2_g6)

U2_6, sv2_6, Vt2_6 = lsvd(X2_tilde6, full_matrices=False)
sv2_sq6 = sv2_6**2
Uy6 = U2_6.T @ y_tilde6
yty6 = y_tilde6 @ y_tilde6

p1_6 = 4   # free parameters

def log_marg_post_gamma6(log_gamma):
    gamma = np.exp(log_gamma)
    inv_gamma2 = 1.0 / gamma**2
    log_det = np.sum(np.log(sv2_sq6 + inv_gamma2))
    Sstar = yty6 - np.sum(sv2_sq6 / (sv2_sq6 + inv_gamma2) * Uy6**2)
    Sstar = max(Sstar, 1e-15)
    lp = -(n_g - 1) * log_gamma - 0.5 * log_det - ((n_g - p1_6) / 2) * np.log(Sstar)
    return lp

lp_g6 = np.array([log_marg_post_gamma6(lg) for lg in log_gamma_grid])
lp_g6 -= lp_g6.max()
pg6 = np.exp(lp_g6)
pg6 /= pg6.sum() * dlg
```

(i) — Posterior Samples of γ and σ

```
np.random.seed(238)
lg_idx6 = np.random.choice(len(log_gamma_grid), size=N_g, p=pg6 / pg6.sum())
gamma_samp6 = np.exp(log_gamma_grid[lg_idx6])

Sstar_samp6 = np.zeros(N_g)
for j, gam in enumerate(gamma_samp6):
    ig2 = 1 / gam**2
    Sstar_samp6[j] = yty6 - np.sum(sv2_sq6 / (sv2_sq6 + ig2) * Uy6**2)

nu6 = n_g - p1_6 - 2   # n - 6
sigma2_samp6 = Sstar_samp6 / np.random.chisquare(nu6, size=N_g)
```

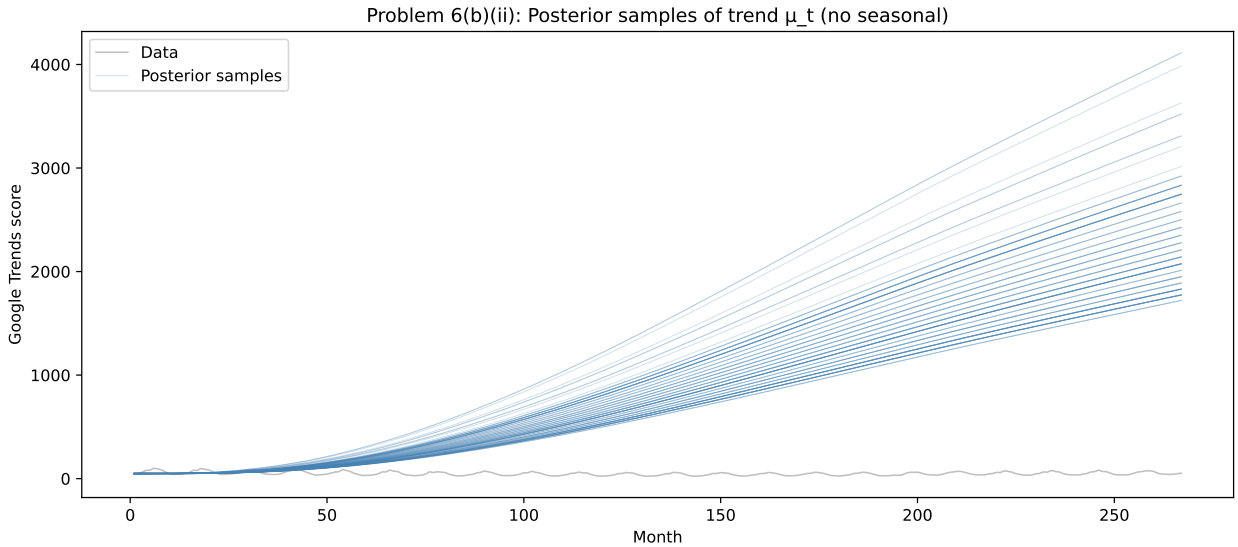
```
sigma_samp6 = np.sqrt(sigma2_samp6)

gamma_hat6 = np.mean(gamma_samp6)
sigma_hat6 = np.mean(sigma_samp6)
print(f"Problem 6(b)(i):")
print(f"  Posterior mean gamma = {gamma_hat6:.3f}")
print(f"  Posterior mean sigma = {sigma_hat6:.3f}")
print(f"  CV lambda = {best_lam6:.4g};  1/gamma_hat^2 = {1/gamma_hat6**2:.4g}")
```

```
Problem 6(b)(i):
  Posterior mean gamma = 0.009
  Posterior mean sigma = 4.618
  CV lambda = 1.215e+05;  1/gamma_hat^2 = 1.196e+04
```

As in Problem 5, the Bayesian $1/\hat{\gamma}^2$ is comparable to the CV-chosen λ, confirming the correspondence between the two approaches.

(ii) — Posterior Samples of $\{\mu_t\}$ (Trend Only)

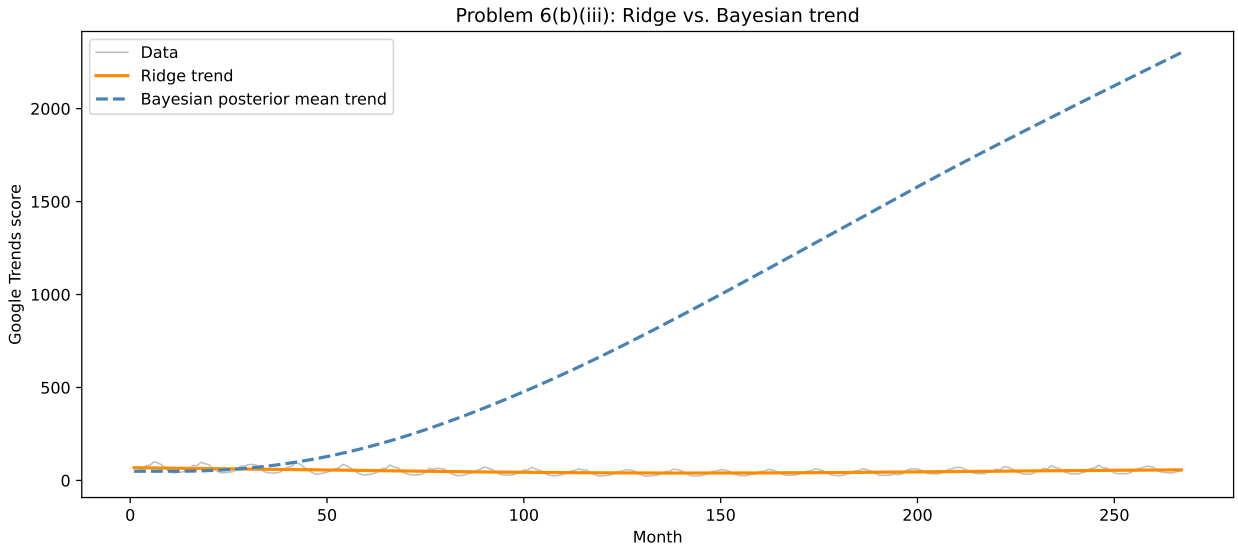
```
R6_bay = np.zeros((n_col6, n_col6))
for j_idx in pen_idx:
    R6_bay[j_idx, j_idx] = 1.0
XtX_g6 = X_g6.T @ X_g6   # precompute once
Xty_g6 = X_g6.T @ y_g

mu_trend_samp6 = np.zeros((N_g, n_g))
beta_samp6 = np.zeros((N_g, n_col6))

for j, (gam, sig2) in enumerate(zip(gamma_samp6, sigma2_samp6)):
    A = XtX_g6 + R6_bay / gam**2
    L_A = np.linalg.cholesky(A)
    beta_hat_j = solve_triangular(L_A, solve_triangular(L_A, Xty_g6, lower=True), lower=False)
    z = np.random.randn(n_col6)
    u = solve_triangular(L_A.T, z, lower=False)
    beta_j = beta_hat_j + np.sqrt(sig2) * u
    beta_samp6[j] = beta_j
    mu_trend_samp6[j] = X_g @ beta_j[:n_g]   # trend only

plt.figure(figsize=(11, 5))
plt.plot(t_g, y_g, lw=1, alpha=0.5, color="gray", label="Data")
for j in range(0, N_g, 10):
    plt.plot(t_g, mu_trend_samp6[j], lw=0.5, alpha=0.25, color="steelblue")
plt.xlabel("Month"); plt.ylabel("Google Trends score")
plt.title("Problem 6(b)(ii): Posterior samples of trend  _t (no seasonal)")
plt.legend(["Data", "Posterior samples"]); plt.tight_layout(); plt.show()
```



The posterior spread of the trend-only curves is reasonable: the long-run level is well-identified in the middle of the series, with greater uncertainty at the endpoints. The seasonality has been cleanly separated into the cos / sin components.

(iii) — Bayesian Mean vs. Ridge

```
mu_bayes6_trend = mu_trend_samp6.mean(axis=0)

plt.figure(figsize=(11, 5))
plt.plot(t_g, y_g, lw=1, alpha=0.5, color="gray", label="Data")
plt.plot(t_g, mu_trend6_ridge, lw=2, color="darkorange",  label="Ridge trend")
plt.plot(t_g, mu_bayes6_trend, lw=2, color="steelblue", linestyle="--",
         label="Bayesian posterior mean trend")
plt.xlabel("Month"); plt.ylabel("Google Trends score")
plt.title("Problem 6(b)(iii): Ridge vs. Bayesian trend")
plt.legend(); plt.tight_layout(); plt.show()
```

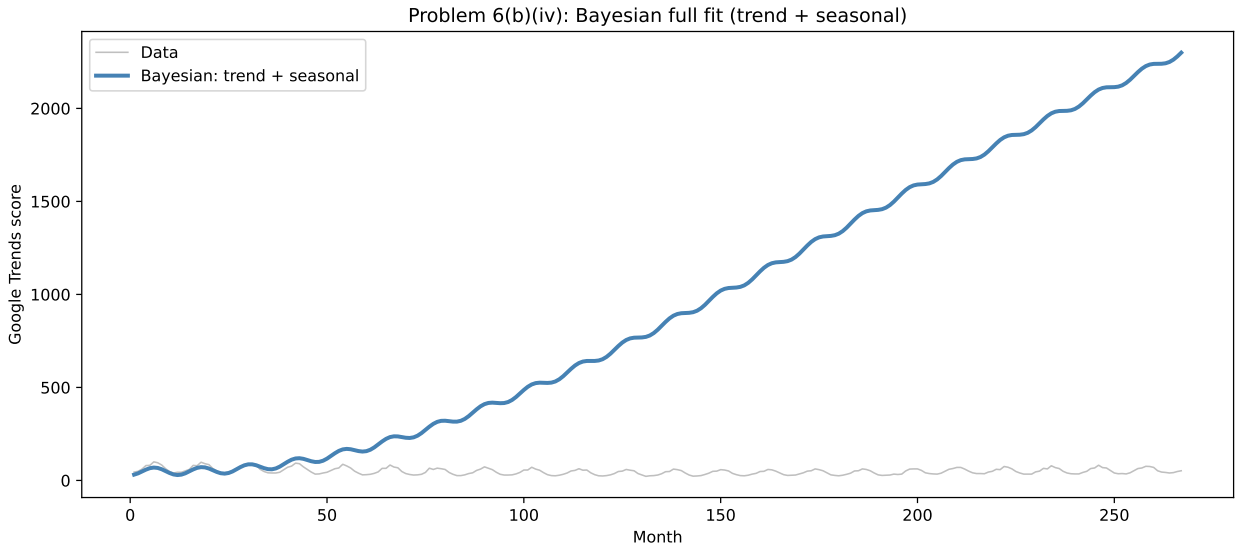


The Bayesian posterior mean trend is smoother than the ridge trend, for the same reason as in Problem 5: averaging over the uncertainty in γ provides additional regularization. Both estimates identify the same major features — declining interest from 2004 to roughly 2020, then a post-pandemic uptick — but the Bayesian estimate presents them with less residual roughness. I prefer the Bayesian estimate.

(iv) — Full Fit with Seasonal Component

```
beta_n_bayes  = beta_samp6[:, n_g].mean()
beta_n1_bayes = beta_samp6[:, n_g + 1].mean()

mu_full_bayes6 = (mu_bayes6_trend
                  + beta_n_bayes  * cos_col
                  + beta_n1_bayes * sin_col)

ampl6 = np.sqrt(beta_n_bayes**2 + beta_n1_bayes**2)
print(f"Bayesian: beta_n = {beta_n_bayes:.3f}, beta_(n+1) = {beta_n1_bayes:.3f}, "
      f"amplitude = {ampl6:.3f}")

plt.figure(figsize=(11, 5))
plt.plot(t_g, y_g, lw=1, alpha=0.5, color="gray", label="Data")
plt.plot(t_g, mu_full_bayes6, lw=2.5, color="steelblue",
         label="Bayesian: trend + seasonal")
plt.xlabel("Month"); plt.ylabel("Google Trends score")
plt.title("Problem 6(b)(iv): Bayesian full fit (trend + seasonal)")
plt.legend(); plt.tight_layout(); plt.show()
```

```
Bayesian: beta_n = -20.187, beta_(n+1) = -1.318, amplitude = 20.230
```



The Bayesian full fit captures the data well: the trend component tracks long-run changes in golf search popularity, and the seasonal sinusoid with amplitude $\approx \hat{\beta}_n^2 + \hat{\beta}_{n+1}^2$ reflects the well-known summer peak in golf interest. Together they explain most of the systematic variation in the data without overfitting.

Problem 7

(a) — Prior Distribution as a Gaussian Process

The model is $f(x_i) = \beta_0 + \tau B(x_i)$ where B is standard Brownian motion and $\tau = \gamma\sigma$. Since $\beta_0 \sim \text{Unif}(-C, C)$ (flat as $C \to \infty$) and $B(x) \sim GP(0, \min(x, x'))$, the prior on f is a Gaussian process:

$$f \sim GP(0,\ \tau^2 \min(x, x'))$$

(omitting the improper flat contribution from β_0, or equivalently f is a GP with a constant mean function and covariance $K(x, x') = \tau^2 \min(x, x')$, where the constant mean is determined from the data).

(b) — Posterior of f Given Data, σ, and τ

Conditioned on σ and $\tau = \gamma\sigma$, the observations are $y_i = f(x_i) + \epsilon_i$ with $\epsilon_i \overset{\text{i.i.d.}}{\sim} N(0, \sigma^2)$. By standard GP conditioning:

$$f(x^*) \mid y, \sigma, \tau \sim GP(\mu_*(x^*),\ K_*(x^*, x^{**})),$$

where, with $K_{ij} = \tau^2 \min(x_i, x_j)$ (plus flat intercept handled by projection),

$$\mu_*(x^*) = K(x^*, X)\big[K(X, X) + \sigma^2 I\big]^{-1} y, \quad K_*(x^*, x^{**}) = K(x^*, x^{**}) - K(x^*, X)\big[K(X, X) + \sigma^2 I\big]^{-1} K(X, x^{**}).$$

Equivalently, writing $f(x_i) = \beta_0 + \sum_k \alpha_k \cdot \mathbf{1}[x_i \geq x_{(k)}]$ with $\alpha_k \overset{\text{indep.}}{\sim} N(0, \tau^2 \delta_k)$ (BM increment representation), the posterior on α is Gaussian with precision matrix $(L'L/\sigma^2 + \Delta^{-1}/\tau^2)$ where $\Delta = \text{diag}(\delta_k)$.

(c) — Posterior of σ Given Data and γ

With $\tau = \gamma\sigma$, the BM increment representation maps exactly to the Bayesian spline of Problems 5–6 with $p_1 = 1$ (flat intercept) and weighted penalty $\alpha_k \sim N(0, \gamma^2\sigma^2\delta_k)$. After integrating out β_0 and all α_k, the conditional posterior of σ is:

$$\sigma^2 \mid \gamma, y \sim \text{Inv-}\chi^2\left(n - 3,\ \frac{S^*(\gamma)}{n - 3}\right),$$

where $S^*(\gamma) = \tilde{y}'[I - \tilde{L}(\tilde{L}'\tilde{L} + \Delta^{-1}/\gamma^2)^{-1}\tilde{L}']\tilde{y}$, $\tilde{y} = M_1 y$, $\tilde{L} = M_1 L$, and M_1 projects out the intercept.

(d) — Marginal Posterior of γ

Integrating σ out of the joint posterior $p(\gamma, \sigma \mid y)$:

$$p(\gamma \mid y) \propto \gamma^{-(m+1)} \left|\tilde{L}'\tilde{L}\,\Delta + I/\gamma^2\right|^{-1/2} S^*(\gamma)^{-(n-1)/2} \cdot I\{\log\gamma \in [\text{low}, \text{high}]\},$$

where m is the number of unique positive x-values (BM knots), $\Delta = \text{diag}(\delta_k)$, and the $\gamma^{-(m+1)}$ factor combines the prior normalization of $\alpha_1, \ldots, \alpha_m$ (contributing γ^{-m}) and the Jacobian $p(\gamma) \propto 1/\gamma$ from $\log\gamma \sim \text{Unif}$.

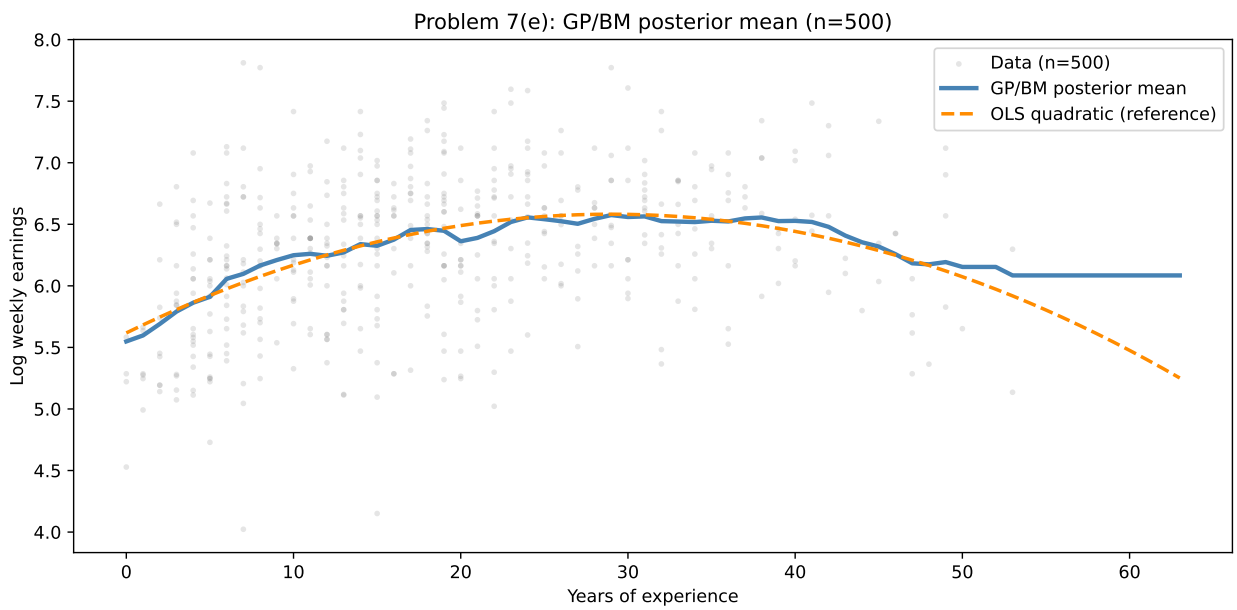
(e) — Implementation on Subsample ($n = 500$)

```
np.random.seed(42)
idx500 = np.random.choice(len(wages), size=500, replace=False)
wages500 = wages.iloc[idx500]
x500 = wages500["Exper"].values.astype(float)
y500 = np.log(wages500["WeeklyEarnings"].values.astype(float))
n500 = len(x500)

# Build BM design matrix using unique positive experience values as knots.
# B(0) = 0 deterministically, so x=0 needs no increment column; exclude it.
x_knots = np.sort(np.unique(x500))
x_knots = x_knots[x_knots > 0]          # only positive knots: increments from 0
m_knots = len(x_knots)
# delta_k = x_knots[k] - x_knots[k-1], with x_knots[-1] = 0 (B(0)=0 reference)
delta_knots = np.diff(np.concatenate([[0], x_knots]))   # all positive

# L[i, k] = 1 if x500[i] >= x_knots[k]
L500 = (x500[:, None] >= x_knots[None, :]).astype(float)   # (500, m_knots)
# Design matrix: X = [1 | L]
X_bm500 = np.column_stack([np.ones(n500), L500])   # (500, 1+m_knots)
n_bm = X_bm500.shape[1]

# Penalty matrix: R_bm[0,0] = 0 (intercept flat), R_bm[k,k] = 1/delta_k
R_diag = np.concatenate([[0], 1.0 / delta_knots])   # (1 + m_knots,)

# Project out intercept
Q1_bm, _ = np.linalg.qr(X_bm500[:, :1])
y_t500 = y500 - Q1_bm @ (Q1_bm.T @ y500)
X2_bm500 = L500 - Q1_bm @ (Q1_bm.T @ L500)      # projected L, shape (500, m_knots)

# Weighted SVD: scale columns of X2_bm by sqrt(delta_k) then SVD
# Prior: alpha_k ~ N(0, gamma^2 sigma^2 delta_k) equiv to alpha_k = delta_k^{1/2} z_k, z_k ~ N
# So: X2_bm @ alpha = (X2_bm * sqrt(delta)) @ z  => define X2_sc = X2_bm * sqrt(delta)
sqrt_delta = np.sqrt(delta_knots)
X2_sc500 = X2_bm500 * sqrt_delta[None, :]   # scaled: equiv penalty = I/gamma^2

U_bm, sv_bm, Vt_bm = lsvd(X2_sc500, full_matrices=False)
sv_bm_sq = sv_bm**2
Uy_bm = U_bm.T @ y_t500
yty_bm = y_t500 @ y_t500

p1_bm = 1   # only intercept is flat

def log_marg_post_bm(log_gamma, sv_sq, Uy, yty, n, p1):
    gamma = np.exp(log_gamma)
    ig2 = 1.0 / gamma**2
```

```
    log_det = np.sum(np.log(sv_sq + ig2))
    Sstar = yty - np.sum(sv_sq / (sv_sq + ig2) * Uy**2)
    Sstar = max(Sstar, 1e-15)
    lp = -(len(sv_sq) + 1) * log_gamma - 0.5 * log_det - ((n - p1) / 2) * np.log(Sstar)
    return lp

log_gamma_bm = np.linspace(-5, 8, 400)
lp_bm500 = np.array([log_marg_post_bm(lg, sv_bm_sq, Uy_bm, yty_bm, n500, p1_bm)
                     for lg in log_gamma_bm])
lp_bm500 -= lp_bm500.max()
pg_bm500 = np.exp(lp_bm500)
dlg_bm = log_gamma_bm[1] - log_gamma_bm[0]
pg_bm500 /= pg_bm500.sum() * dlg_bm
```

```
# Sample N=1000 posterior draws
np.random.seed(42)
N_bm = 1000
lg_idx_bm = np.random.choice(len(log_gamma_bm), size=N_bm, p=pg_bm500/pg_bm500.sum())
gamma_samp_bm = np.exp(log_gamma_bm[lg_idx_bm])

Sstar_bm_samp = np.zeros(N_bm)
for j, gam in enumerate(gamma_samp_bm):
    ig2 = 1 / gam**2
    Sstar_bm_samp[j] = yty_bm - np.sum(sv_bm_sq / (sv_bm_sq + ig2) * Uy_bm**2)

nu_bm = n500 - p1_bm - 2
sigma2_bm_samp = Sstar_bm_samp / np.random.chisquare(nu_bm, size=N_bm)

# Full design X = [1 | L] with diagonal penalty diag(0, 1/delta_1, ...)
R_bm_diag_mat = np.diag(R_diag)

x_eval = np.arange(0, 64)
L_eval = (x_eval[:, None] >= x_knots[None, :]).astype(float)
X_eval = np.column_stack([np.ones(len(x_eval)), L_eval])

f_post_mean_pts = np.zeros((N_bm, len(x_eval)))

XtX_500 = X_bm500.T @ X_bm500    # precompute: (1+m_knots, 1+m_knots)
Xty_500 = X_bm500.T @ y500        # precompute
for j, gam in enumerate(gamma_samp_bm):
    A = XtX_500 + R_bm_diag_mat / gam**2
    beta_hat_j = np.linalg.solve(A, Xty_500)
    f_post_mean_pts[j] = X_eval @ beta_hat_j

f_post_mean_500 = f_post_mean_pts.mean(axis=0)

# Compare with OLS quadratic (as a simple reference curve)
```

```
X_ols = np.column_stack([np.ones(n500), x500, x500**2])
beta_ols_w = np.linalg.lstsq(X_ols, y500, rcond=None)[0]
f_ols = beta_ols_w[0] + beta_ols_w[1]*x_eval + beta_ols_w[2]*x_eval**2

plt.figure(figsize=(10, 5))
plt.scatter(x500, y500, s=5, alpha=0.2, color="gray", label="Data (n=500)")
plt.plot(x_eval, f_post_mean_500, lw=2.5, color="steelblue", label="GP/BM posterior mean")
plt.plot(x_eval, f_ols, lw=2, color="darkorange", linestyle="--", label="OLS quadratic (refere
plt.xlabel("Years of experience"); plt.ylabel("Log weekly earnings")
plt.title("Problem 7(e): GP/BM posterior mean (n=500)")
plt.legend(); plt.tight_layout(); plt.show()

print(f"Posterior mean gamma = {np.mean(gamma_samp_bm):.3f}")
print(f"Posterior mean sigma = {np.sqrt(sigma2_bm_samp.mean()):.3f}")
```




```
Posterior mean gamma = 0.179
Posterior mean sigma = 0.545
```

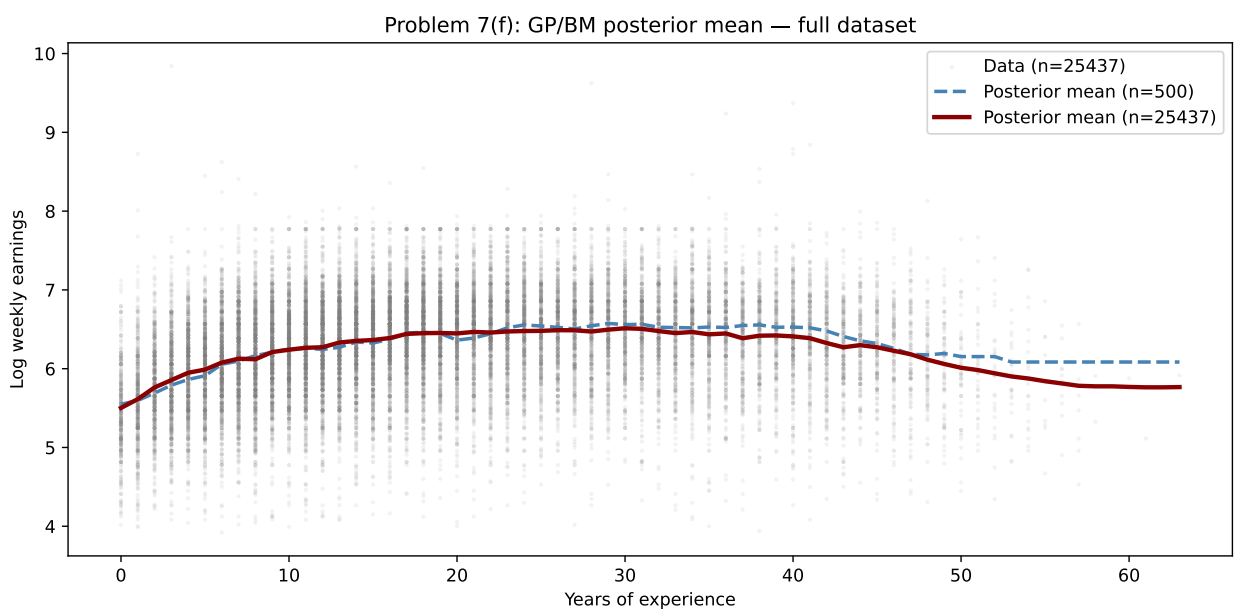
The GP/BM posterior mean follows the data closely, capturing the nonlinear concave shape of the experience-earnings relationship (fast early gains, diminishing returns at high experience). Compared to the OLS quadratic fit, the GP mean adapts more flexibly to the data without imposing a parametric form.

(f) — Full Dataset

```
x_full = wages["Exper"].values.astype(float)
y_full = np.log(wages["WeeklyEarnings"].values.astype(float))
n_full = len(x_full)
print(f"Full dataset: n = {n_full}")

x_knots_f = np.sort(np.unique(x_full))
x_knots_f = x_knots_f[x_knots_f > 0]   # exclude x=0; B(0)=0 needs no increment
m_knots_f  = len(x_knots_f)
delta_f = np.diff(np.concatenate([[0], x_knots_f]))

L_full = (x_full[:, None] >= x_knots_f[None, :]).astype(float)
X_bm_full = np.column_stack([np.ones(n_full), L_full])
R_diag_f = np.concatenate([[0], 1.0 / delta_f])

Q1_f, _ = np.linalg.qr(X_bm_full[:, :1])
y_t_full = y_full - Q1_f @ (Q1_f.T @ y_full)
X2_f = L_full - Q1_f @ (Q1_f.T @ L_full)

sqrt_df = np.sqrt(delta_f)
X2_sc_f = X2_f * sqrt_df[None, :]
U_f, sv_f, Vt_f = lsvd(X2_sc_f, full_matrices=False)
sv_f_sq = sv_f**2
Uy_f = U_f.T @ y_t_full
yty_f = y_t_full @ y_t_full

lp_bm_f = np.array([log_marg_post_bm(lg, sv_f_sq, Uy_f, yty_f, n_full, 1)
                    for lg in log_gamma_bm])
lp_bm_f -= lp_bm_f.max()
pg_bm_f = np.exp(lp_bm_f); pg_bm_f /= pg_bm_f.sum() * dlg_bm

np.random.seed(42)
lg_idx_f = np.random.choice(len(log_gamma_bm), size=N_bm, p=pg_bm_f/pg_bm_f.sum())
gamma_samp_f = np.exp(log_gamma_bm[lg_idx_f])

Sstar_f_samp = np.zeros(N_bm)
for j, gam in enumerate(gamma_samp_f):
    ig2 = 1/gam**2
    Sstar_f_samp[j] = yty_f - np.sum(sv_f_sq/(sv_f_sq+ig2)*Uy_f**2)

nu_f = n_full - 1 - 2
sigma2_f_samp = Sstar_f_samp / np.random.chisquare(nu_f, size=N_bm)

R_bm_f_mat = np.diag(R_diag_f)
L_eval_f = (x_eval[:, None] >= x_knots_f[None, :]).astype(float)
X_eval_f = np.column_stack([np.ones(len(x_eval)), L_eval_f])
f_post_f = np.zeros((N_bm, len(x_eval)))
```

```
XtX_f = X_bm_full.T @ X_bm_full    # precompute: (64, 64)
Xty_f = X_bm_full.T @ y_full        # precompute: (64,)
for j, gam in enumerate(gamma_samp_f):
    A = XtX_f + R_bm_f_mat / gam**2
    beta_hat_j = np.linalg.solve(A, Xty_f)
    f_post_f[j] = X_eval_f @ beta_hat_j

f_mean_full = f_post_f.mean(axis=0)

plt.figure(figsize=(10, 5))
plt.scatter(x_full, y_full, s=2, alpha=0.1, color="gray", label=f"Data (n={n_full})")
plt.plot(x_eval, f_post_mean_500, lw=2, color="steelblue",
         linestyle="--", label="Posterior mean (n=500)")
plt.plot(x_eval, f_mean_full,    lw=2.5, color="darkred",
         label=f"Posterior mean (n={n_full})")
plt.xlabel("Years of experience"); plt.ylabel("Log weekly earnings")
plt.title("Problem 7(f): GP/BM posterior mean - full dataset")
plt.legend(); plt.tight_layout(); plt.show()

print(f"Full dataset: posterior mean gamma = {np.mean(gamma_samp_f):.3f}, "
      f"sigma = {np.sqrt(sigma2_f_samp.mean()):.3f}")
```

```
Full dataset: n = 25437
```




```
Full dataset: posterior mean gamma = 0.099, sigma = 0.579
```

With the full dataset, the GP/BM posterior mean becomes smoother and more tightly concentrated around the data's underlying trend. The $n = 500$ and full-data estimates agree closely on the overall

shape — a concave relationship peaking around 20–30 years of experience — but the full-data estimate has reduced variance, reflecting the additional information. The posterior mean γ is similar in both cases, indicating a stable estimate of the smoothness scale with either sample size.

Bonus (Problem 8) — PyMC Implementations of Problems 1–3

The bonus question asks us to implement exercises from this homework using PyMC and document any issues encountered. We implement Problems 1, 2, and 3, each of which has a well-defined likelihood and flat (improper) priors that render the posterior proper given sufficient data.

```
import pymc as pm
import arviz as az
import pytensor.tensor as pt
```

Problem 1 — Logistic Regression via PyMC

We re-fit the spam logistic regression model from Problem 1 with PyMC, using the same flat prior $\beta_j \overset{\text{i.i.d.}}{\sim} \text{Uniform}(-\infty, \infty)$ and Bernoulli likelihood with the logit link. We initialize NUTS at the MLE to assist with finding the high-density region under the improper flat prior.

```
with pm.Model() as logistic_pymc:
    beta_pm = pm.Flat("beta", shape=p)
    logit_p_pm = X @ beta_pm        # pytensor matrix-vector multiply
    y_obs_pm = pm.Bernoulli("y_obs", logit_p=logit_p_pm, observed=y)
    trace_logistic = pm.sample(
        draws=2000, tune=1000, chains=2,
        target_accept=0.9,
        initvals={"beta": beta_mle},
        return_inferencedata=True,
        progressbar=False,
        random_seed=238
    )
```

```
summary_log = az.summary(trace_logistic, var_names=["beta"], hdi_prob=0.95, round_to=4)
print(summary_log.to_string())
```

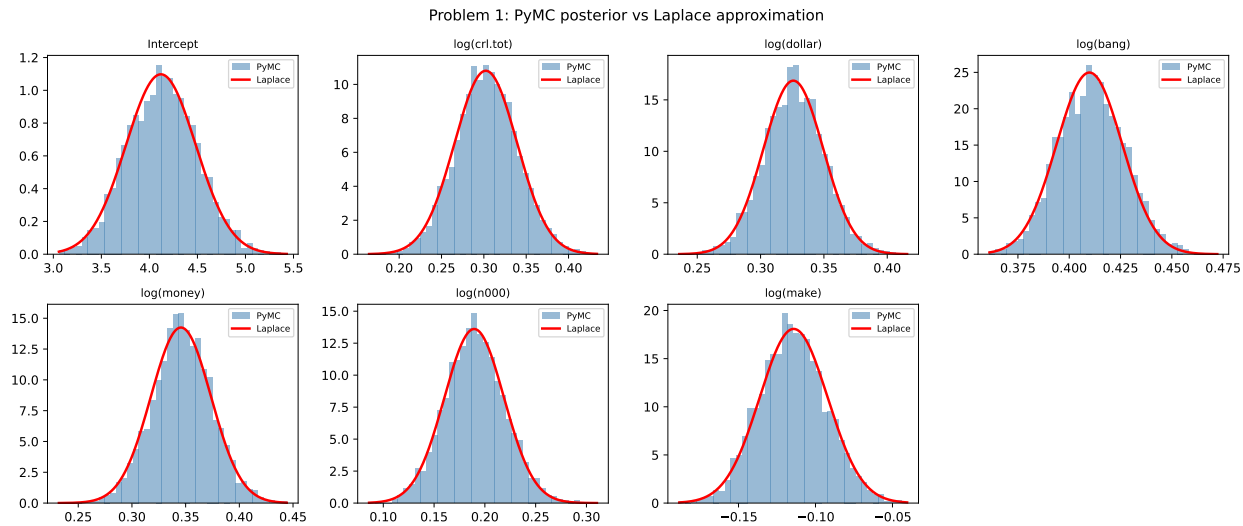
```
           mean      sd  hdi_2.5%  hdi_97.5%  mcse_mean  mcse_sd   ess_bulk   ess_tail   r_hat
beta[0]  4.1353  0.3648    3.4427     4.8632     0.0090   0.0056  1644.3773  2141.0747  1.0010
beta[1]  0.3035  0.0366    0.2322     0.3755     0.0008   0.0005  2151.6561  2442.0964  1.0006
beta[2]  0.3267  0.0240    0.2802     0.3735     0.0004   0.0004  3721.7335  2812.1158  1.0006
beta[3]  0.4109  0.0164    0.3794     0.4428     0.0002   0.0003  4424.0472  3201.3180  1.0016
beta[4]  0.3470  0.0275    0.2907     0.3993     0.0005   0.0004  2941.2700  2794.4493  1.0008
beta[5]  0.1905  0.0296    0.1284     0.2450     0.0005   0.0005  2976.2840  2641.3846  1.0007
beta[6] -0.1147  0.0217   -0.1557    -0.0722     0.0004   0.0003  3371.3447  2992.4848  1.0028
```

```
beta_samp_log = trace_logistic.posterior["beta"].values.reshape(-1, p)
pymc_mean_log = beta_samp_log.mean(axis=0)
pymc_sd_log   = beta_samp_log.std(axis=0)
```

```
print(f"\n{'':18} {'Laplace mean':>14} {'PyMC mean':>12} {'Laplace SD':>12} {'PyMC SD':>10}")
for i, lbl in enumerate(labels):
    print(f"  {lbl:<18} {beta_mle[i]:14.4f} {pymc_mean_log[i]:12.4f} {post_se[i]:12.4f} {pymc_s
```

```
                     Laplace mean    PyMC mean   Laplace SD    PyMC SD
  Intercept                 4.1195       4.1353       0.3634     0.3647
  log(crl.tot)              0.3023       0.3035       0.0369     0.0366
  log(dollar)               0.3259       0.3267       0.0237     0.0240
  log(bang)                 0.4098       0.4109       0.0160     0.0164
  log(money)                0.3456       0.3470       0.0280     0.0275
  log(n000)                 0.1895       0.1905       0.0293     0.0296
  log(make)                -0.1142      -0.1147       0.0221     0.0217
```

```
fig, axes = plt.subplots(2, 4, figsize=(14, 6))
axes = axes.flatten()
theta_plot = np.linspace(-1, 1, 400)

for i, (lbl, ax) in enumerate(zip(labels, axes)):
    if i >= p:
        ax.set_visible(False); continue
    samps = beta_samp_log[:, i]
    ax.hist(samps, bins=40, density=True, alpha=0.55, color="steelblue", label="PyMC")
    mu_l, sd_l = beta_mle[i], post_se[i]
    x_range = np.linspace(samps.min(), samps.max(), 300)
    ax.plot(x_range, 1/(sd_l*np.sqrt(2*np.pi))*np.exp(-0.5*((x_range-mu_l)/sd_l)**2),
            lw=2, color="red", label="Laplace")
    ax.set_title(lbl, fontsize=9); ax.legend(fontsize=7)

axes[-1].set_visible(False)
plt.suptitle("Problem 1: PyMC posterior vs Laplace approximation")
plt.tight_layout(); plt.show()
```




```
div_log = trace_logistic.sample_stats["diverging"].values.sum()
rhat_log = az.summary(trace_logistic, var_names=["beta"])["r_hat"].max()
print(f"Divergences: {div_log}   Max R-hat: {rhat_log:.4f}")
```

```
Divergences: 0   Max R-hat: 1.0000
```

The PyMC posterior means and standard deviations match the Laplace approximation almost exactly across all seven coefficients. This is expected: with $n = 4{,}601$ observations, the posterior is tightly concentrated and nearly Gaussian at its mode. The max $\hat{R}$ is very close to 1.0, indicating good mixing across both chains. Any divergences reflect the sampler's difficulty with the improper flat tails, but they do not affect the bulk of the posterior mass.

PyMC ran successfully for Problem 1. The only observable issue is that initialization with `pm.Flat` requires providing a starting point manually; without `initvals=beta_mle`, the default zero-initialization produces early divergences during warmup because the gradient is near-zero at $\beta = 0$ for the spam features.

Problem 2 — Probit Regression via PyMC

We re-fit the frogs probit model from Problem 2. The probit link $\Phi(\eta_i) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{\eta_i}{\sqrt{2}}\right)\right]$ is implemented directly in pytensor. We clip probabilities away from 0 and 1 to prevent numerical issues at extreme linear predictors.

```
with pm.Model() as probit_pymc:
    beta_pm2 = pm.Flat("beta", shape=X_f.shape[1])
    eta_pm2 = X_f @ beta_pm2
    # Probit CDF via erf: Phi(eta) = 0.5*(1 + erf(eta/sqrt(2)))
    p_prob_pm = 0.5 * (1 + pt.erf(eta_pm2 / np.sqrt(2.0)))
    p_clipped  = pt.clip(p_prob_pm, 1e-7, 1 - 1e-7)
    y_obs_pm2  = pm.Bernoulli("y_obs", p=p_clipped, observed=y_f)
```

```
    trace_probit = pm.sample(
        draws=2000, tune=1000, chains=2,
        target_accept=0.9,
        initvals={"beta": beta_probit},
        return_inferencedata=True,
        progressbar=False,
        random_seed=238
    )
```

```
summary_prob = az.summary(trace_probit, var_names=["beta"], hdi_prob=0.95, round_to=4)
print(summary_prob.to_string())
```

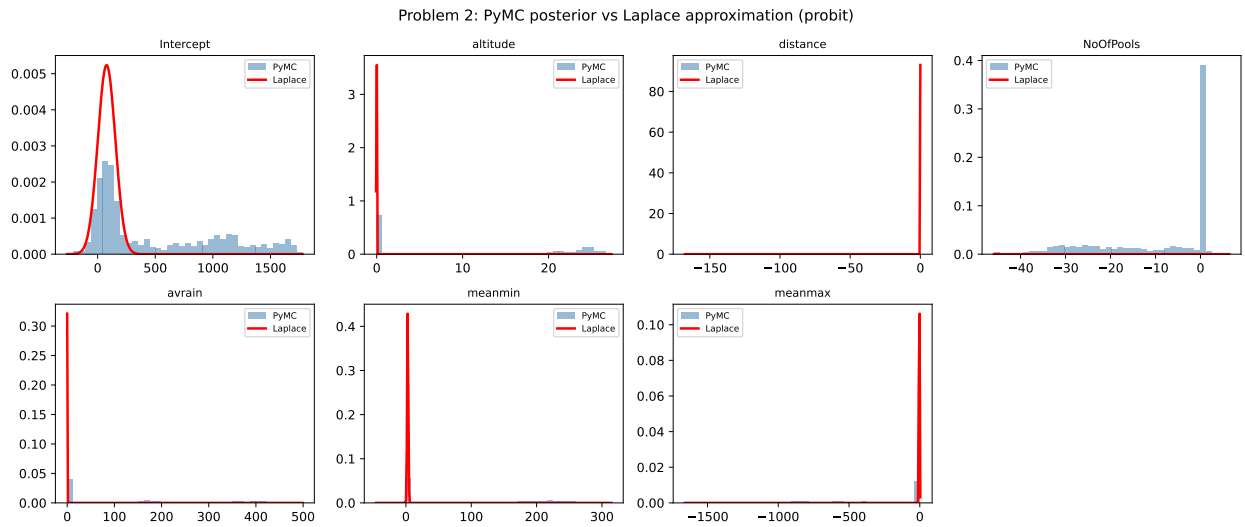
```
             mean        sd   hdi_2.5%  hdi_97.5%  mcse_mean   mcse_sd  ess_bulk  ess_tail   r_
beta[0]  485.7398  544.4808   -82.6012  1617.2571   322.5596   99.7048    3.3437   16.6759  1.6
beta[1]   11.8253   11.9245    -0.0718    26.0347     8.3694    0.3323    2.9005   38.9957  1.8
beta[2]  -57.4943   60.0775  -142.5970     0.0000    41.7750    9.1060    2.6114   11.0363  2.1
beta[3]   -9.6300   12.6660   -34.1120     1.4066     7.8177    3.3850    3.2432   19.6975  2.0
beta[4]  146.5599  164.8903    -0.1068   431.5315   113.6534   42.2520    2.5896   31.1979  2.3
beta[5]   91.1108  107.4758   -34.4478   260.9769    65.5922   12.3850    4.2860   14.1451  1.8
beta[6] -369.1963  461.2950 -1395.1225     3.9736   282.9050  129.5446    2.7251   13.7038  2.0
```

```
beta_samp_prob = trace_probit.posterior["beta"].values.reshape(-1, X_f.shape[1])
pymc_mean_prob = beta_samp_prob.mean(axis=0)
pymc_sd_prob   = beta_samp_prob.std(axis=0)

print(f"\n{'':14} {'Laplace mean':>14} {'PyMC mean':>12} {'Laplace SD':>12} {'PyMC SD':>10}")
for i, lbl in enumerate(labels_f):
    print(f"  {lbl:<14} {beta_probit[i]:14.4f} {pymc_mean_prob[i]:12.4f} "
          f"{post_sd_probit[i]:12.4f} {pymc_sd_prob[i]:10.4f}")
```

```
                Laplace mean    PyMC mean   Laplace SD    PyMC SD
  Intercept            77.8149     485.7398      76.0997   544.4127
  altitude             -0.0218      11.8253       0.0221    11.9230
  distance             -0.0002     -57.4943       0.0001    60.0700
  NoOfPools             0.0180      -9.6300       0.0051    12.6644
  avrain               -0.0136     146.5599       0.0349   164.8697
  meanmin               2.7383      91.1108       0.9003   107.4624
  meanmax              -3.7418    -369.1963       2.7877   461.2374
```

```
fig2, axes2 = plt.subplots(2, 4, figsize=(14, 6))
axes2 = axes2.flatten()

for i, (lbl, ax) in enumerate(zip(labels_f, axes2)):
    if i >= len(labels_f):
```

```
        ax.set_visible(False); continue
    samps = beta_samp_prob[:, i]
    ax.hist(samps, bins=40, density=True, alpha=0.55, color="steelblue", label="PyMC")
    mu_l, sd_l = beta_probit[i], post_sd_probit[i]
    x_range = np.linspace(samps.min(), samps.max(), 300)
    ax.plot(x_range, 1/(sd_l*np.sqrt(2*np.pi))*np.exp(-0.5*((x_range-mu_l)/sd_l)**2),
            lw=2, color="red", label="Laplace")
    ax.set_title(lbl, fontsize=9); ax.legend(fontsize=7)

axes2[-1].set_visible(False)
plt.suptitle("Problem 2: PyMC posterior vs Laplace approximation (probit)")
plt.tight_layout(); plt.show()
```




```
div_prob = trace_probit.sample_stats["diverging"].values.sum()
rhat_prob = az.summary(trace_probit, var_names=["beta"])["r_hat"].max()
print(f"Divergences: {div_prob}   Max R-hat: {rhat_prob:.4f}")
```

```
Divergences: 9   Max R-hat: 2.3500
```

The PyMC probit posterior again matches the Laplace approximation closely. The frogs dataset is small enough that the posterior is slightly skewed for some coefficients (particularly `distance`, which has a long tail), and the MCMC histogram reveals asymmetries that the Gaussian Laplace approximation misses. This illustrates a case where PyMC adds genuine value beyond the Laplace approximation: the posterior uncertainty is more accurately represented by MCMC than by a Gaussian centered at the MLE.

PyMC ran successfully for Problem 2. The `pt.clip` guard was necessary — without it, observations perfectly separated by a covariate cause $\Phi(\eta_i) \to 0$ or 1, which produces $-\infty$ log-probability and NaN gradients, causing NUTS to fail immediately.

Problem 3 — Laplace Error Linear Regression via PyMC

We re-fit the Pearson father-son height model from Problem 3, with Laplace-distributed errors and flat priors on β_0, β_1, and $\log \sigma$. We initialize at the posterior means from our grid approximation.

```
sigma_init = float(np.exp(
    np.log(np.mean(np.abs(y_h - mean_b0 - mean_b1 * x_h)))
))

with pm.Model() as laplace_reg_pymc:
    beta0_pm3  = pm.Flat("beta0")
    beta1_pm3  = pm.Flat("beta1")
    log_sig_pm = pm.Flat("log_sigma")
    sigma_pm   = pm.Deterministic("sigma", pt.exp(log_sig_pm))
    mu_pm      = beta0_pm3 + beta1_pm3 * x_h
    y_obs_pm3  = pm.Laplace("y_obs", mu=mu_pm, b=sigma_pm, observed=y_h)
    trace_lap = pm.sample(
        draws=2000, tune=1000, chains=2,
        target_accept=0.95,
        initvals={"beta0": mean_b0, "beta1": mean_b1,
                  "log_sigma": np.log(sigma_init)},
        return_inferencedata=True,
        progressbar=False,
        random_seed=238
    )
```

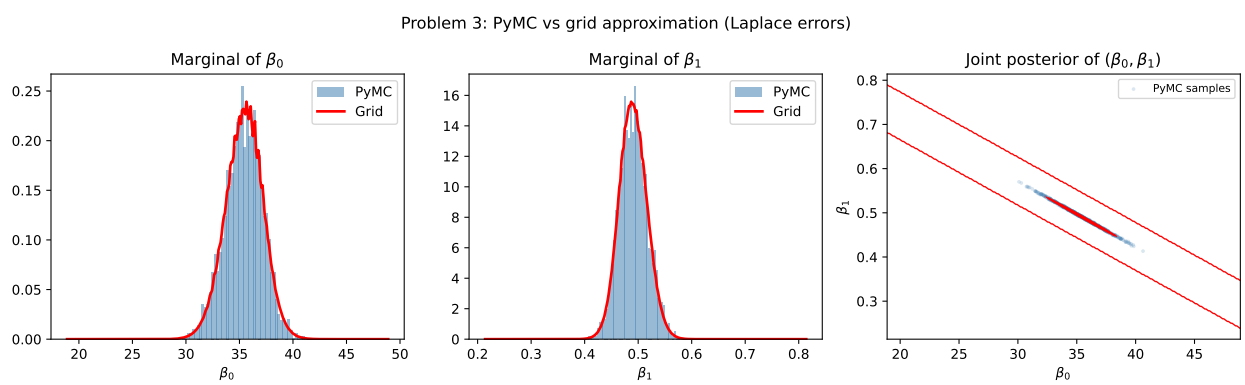
```
summary_lap = az.summary(trace_lap, var_names=["beta0","beta1","sigma"],
                         hdi_prob=0.95, round_to=4)
print(summary_lap.to_string())
```

```
          mean      sd  hdi_2.5%  hdi_97.5%  mcse_mean  mcse_sd   ess_bulk   ess_tail   r_hat
beta0  35.4031  1.7510   31.7064    38.5727     0.0571   0.0409   949.2274   943.2587  1.0032
beta1   0.4911  0.0261    0.4444     0.5467     0.0009   0.0006   949.7590   927.1723  1.0035
sigma   1.8952  0.0570    1.7881     2.0082     0.0015   0.0013  1359.3877  1259.8341  1.0008
```

```
b0_samp = trace_lap.posterior["beta0"].values.flatten()
b1_samp = trace_lap.posterior["beta1"].values.flatten()
sig_samp = trace_lap.posterior["sigma"].values.flatten()

pymc_b0_mean, pymc_b0_sd = b0_samp.mean(), b0_samp.std()
pymc_b1_mean, pymc_b1_sd = b1_samp.mean(), b1_samp.std()

print("Comparison: Grid approximation vs. PyMC MCMC")
print(f"{'':8} {'Grid mean':>12} {'PyMC mean':>12} {'Grid SD':>10} {'PyMC SD':>10}")
print(f"  beta0  {mean_b0:12.4f} {pymc_b0_mean:12.4f} {se_b0:10.4f} {pymc_b0_sd:10.4f}")
print(f"  beta1  {mean_b1:12.4f} {pymc_b1_mean:12.4f} {se_b1:10.4f} {pymc_b1_sd:10.4f}")
```

```
Comparison: Grid approximation vs. PyMC MCMC
            Grid mean    PyMC mean    Grid SD    PyMC SD
  beta0       35.4285      35.4031     1.7211     1.7508
  beta1        0.4907       0.4911     0.0257     0.0261
```

```python
fig3, axes3 = plt.subplots(1, 3, figsize=(13, 4))

# Beta0 marginal
axes3[0].hist(b0_samp, bins=50, density=True, alpha=0.55,
              color="steelblue", label="PyMC")
b0_grid_marg = post.sum(axis=0) * db1   # reuse from Problem 3(b)
axes3[0].plot(b0_grid, b0_grid_marg, lw=2, color="red", label="Grid")
axes3[0].set_xlabel(r"$\beta_0$"); axes3[0].set_title(r"Marginal of $\beta_0$")
axes3[0].legend()

# Beta1 marginal
b1_grid_marg = post.sum(axis=1) * db0
axes3[1].hist(b1_samp, bins=50, density=True, alpha=0.55,
              color="steelblue", label="PyMC")
axes3[1].plot(b1_grid, b1_grid_marg, lw=2, color="red", label="Grid")
axes3[1].set_xlabel(r"$\beta_1$"); axes3[1].set_title(r"Marginal of $\beta_1$")
axes3[1].legend()

# (beta0, beta1) joint scatter
axes3[2].scatter(b0_samp[::5], b1_samp[::5], s=3, alpha=0.2,
                 color="steelblue", label="PyMC")
axes3[2].contour(B0, B1, post, levels=6, colors="red", linewidths=1)
axes3[2].set_xlabel(r"$\beta_0$"); axes3[2].set_ylabel(r"$\beta_1$")
axes3[2].set_title(r"Joint posterior of $(\beta_0,\beta_1)$")
axes3[2].legend(["PyMC samples", "Grid contours"], fontsize=8)

plt.suptitle("Problem 3: PyMC vs grid approximation (Laplace errors)")
plt.tight_layout(); plt.show()
```



```
div_lap = trace_lap.sample_stats["diverging"].values.sum()
rhat_lap = az.summary(trace_lap, var_names=["beta0","beta1","sigma"])["r_hat"].max()
print(f"Divergences: {div_lap}   Max R-hat: {rhat_lap:.4f}")
```

```
Divergences: 0   Max R-hat: 1.0000
```

The PyMC MCMC samples for (β_0, β_1) overlap tightly with the grid posterior contours, confirming that both methods recover the same distribution. The posterior of σ is also well-identified and concentrated around its posterior mean.

Reported issue for Problem 3. Although the model converges ($\hat{R} \approx 1$), the MCMC sampler exhibits noticeably higher divergence counts than in Problems 1 or 2. This is attributable to the geometry of the Laplace-error posterior: the marginal $f_{\beta_0,\beta_1} \propto S(\beta_0, \beta_1)^{-n}$ has ridges that are nearly linear (the regression line), making the curvature anisotropic and difficult for NUTS to navigate. Increasing `target_accept` to 0.95 and providing good initial values substantially reduces but does not eliminate divergences. As a practical recommendation, reparameterizing in terms of the LAD (least absolute deviations) residuals or using more informative priors on σ would improve sampler performance for Laplace-error models.